



UNIVERSITÀ DEGLI STUDI ROMA TRE

Dipartimento di Ingegneria Civile, Informatica
e delle Tecnologie Aeronautiche
Corso di Laurea in Ingegneria Informatica

Tesi di Laurea

Metodi di Rigging Automatico, Skinning e Retargeting di Animazioni per Oggetti 3D Inanimati in Blender

Laureando

Jesus Anthony Shuan Arce

Matricola 533954

Relatore

Prof. Franco Milicchio

Anno Accademico 2024/2025

Ringraziamenti

Siamo arrivati alla fine. Se ripenso al primo giorno di università, mi sembra passato un secolo: paure, aspettative, entusiasmo e mille domande su come sarebbe andata. In mezzo ci sono stati esami che sembravano impossibili, giornate infinite sui libri, momenti di sconforto... ma anche tante soddisfazioni, risate e persone che hanno reso questo percorso qualcosa di molto più grande di un semplice titolo di studio.

Grazie al mio relatore per la disponibilità, i consigli e la pazienza durante la stesura di questo lavoro.

Grazie alla mia famiglia, che mi ha supportato (e sopportato) in ogni fase: nelle sessioni infinite, nei “non ce la farò mai”. Senza di voi tutto questo non sarebbe stato lo stesso.

Grazie agli amici di sempre e a quelli conosciuti in questi anni: per gli appunti condivisi all’ultimo minuto, per le pause caffè diventate terapie di gruppo, per le risate nei momenti più stressanti e per aver trasformato l’università in un’esperienza da ricordare.

E infine grazie a me, per aver resistito, per non aver mollato quando sarebbe stato più facile farlo, e per aver continuato a crederci anche nei giorni più complicati.

Questo è solo un punto di partenza. E nel caso non vi rivedessi... buon pomeriggio, buonasera e buonanotte!

Introduzione

L'animazione digitale rappresenta uno dei campi in cui la tecnologia mostra con maggiore evidenza la propria capacità di simulare il movimento e la realtà. Dietro ogni modello 3D animato si trovano concetti e strumenti propri dell'informatica, come algoritmi, calcoli geometrici e trasformazioni matematiche che consentono di rappresentare movimenti credibili e coerenti nello spazio tridimensionale.

L'animazione di un modello digitale richiede una serie di passaggi fondamentali che consentono di trasformare un oggetto tridimensionale statico in un elemento capace di muoversi in modo credibile. Si parte dalla definizione di una struttura interna che ne guida i movimenti, per poi stabilire la relazione tra questa struttura e la superficie del modello. Infine, vengono applicate le sequenze di movimento che danno vita all'oggetto all'interno della scena virtuale. Queste fasi, pur diverse tra loro, cooperano per rendere il comportamento del modello coerente e realistico, permettendo di ottenere animazioni fluide e naturali.

Negli ultimi anni, l'introduzione di tecniche di automazione e di algoritmi avanzati ha reso possibile generare e trasferire animazioni in modo sempre più efficiente. In questo contesto, il presente lavoro si concentra sull'analisi e l'implementazione di metodi per la gestione automatica del rigging e dello skinning, con l'obiettivo di estendere tali processi anche a modelli non convenzionali e oggetti di natura non umanoide.

L'obiettivo generale è quello di esplorare i principi teorici e le soluzioni pratiche che permettono di applicare in modo coerente un movimento realistico a qualsiasi modello tridimensionale, evidenziando le potenzialità e i limiti delle attuali tecniche di animazione automatizzata. La stesura di questa tesi, scritta in LaTeX con il metodo top-down, è articolata come segue:

1. Il primo capitolo introduce i concetti fondamentali della rappresentazione e dell'animazione di modelli tridimensionali. Vengono illustrati i principi di costruzione delle mesh, la gestione di vertici e facce e le principali trasformazioni geometriche nello spazio 3D. Si approfondiscono inoltre i meccanismi di rigging, skinning e animazione scheletrica, ponendo le basi per comprendere le fasi operative successive.
2. Nel secondo capitolo vengono descritti gli strumenti e le tecnologie utilizzate per la realizzazione del progetto. Si analizzano le caratteristiche dei software di modellazione e animazione, i formati di scambio e le modalità di integrazione tra ambienti diversi, delineando il contesto tecnico all'interno del quale si è svolto il lavoro sperimentale.
3. Il terzo capitolo illustra l'approccio seguito per la creazione e l'animazione dei modelli tridimensionali. Vengono descritte le fasi di generazione automatica dello scheletro, l'associazione delle mesh al rig e il processo di trasferimento delle animazioni. La sezione analizza inoltre le scelte metodologiche adottate e le soluzioni sviluppate per ottimizzare il workflow di animazione.
4. Il quarto capitolo è dedicato alla parte sperimentale. Qui vengono presentate le applicazioni pratiche delle tecniche descritte, con particolare attenzione ai risultati ottenuti su modelli non convenzionali. L'analisi dei test evidenzia le prestazioni del sistema, i principali limiti riscontrati e le potenzialità dell'approccio proposto.
5. L'ultimo capitolo raccoglie le conclusioni del lavoro, riassumendo gli obiettivi raggiunti e le criticità incontrate. Vengono inoltre proposte possibili evoluzioni del progetto e nuovi ambiti di applicazione, con l'intento di migliorare ulteriormente l'automazione e la qualità dell'animazione digitale.

Indice

Introduzione	iii
Indice	v
Elenco delle figure	viii
1 Background metodologico	1
1.1 Mesh 3D e rappresentazioni geometriche	1
1.1.1 Vertici, lati, facce e struttura topologica	2
1.1.2 Coordinate locali e globali, trasformazioni con matrici 4x4	2
1.1.3 Differenze tra mesh statica e mesh animata	3
1.2 Trasformazioni geometriche	4
1.2.1 Traslazione, rotazione e scala 3D: rappresentazione matriciale e composizione	5
1.2.2 Rotazione con quaternioni: motivazione e vantaggi rispetto agli Euler angles (no gimbal lock)	8
1.3 Rigging e Skinning	12
1.3.1 Concetto di scheletro e relazione con la mesh	12
1.3.2 Linear Blend Skinning (LBS): formulazione matematica e limiti .	14
1.3.3 Weight Painting e associazione ossa-vertici	16
1.4 Retargeting dell'animazione	17
1.4.1 Concetto di motion retargeting tra modelli diversi	18
1.4.2 Introduzione ai metodi automatici: UniRig e ASMR come ap- procci moderni	18

2	Background tecnologico	20
2.1	Blender	20
2.1.1	Modellazione, rigging e animazione	21
2.1.2	Strumenti per scripting in Python (bpy)	23
2.2	Mixamo	24
2.2.1	Esportazione in formato FBX e compatibilità con Blender e Unity	24
2.3	MoveAI	27
2.3.1	Generazione automatica di animazioni realistiche da video (MoveAI)	27
2.4	Unity 3D	28
2.4.1	Gestione dei modelli animati in Unity	29
2.4.2	Pipeline di importazione FBX → Unity	30
2.4.3	Sistema di retargeting delle animazioni: Animator Controller . .	30
3	Approccio e metodologia	32
3.1	Importazione dei modelli umanoidi	33
3.1.1	Tipologie di modelli (Mixamo, Unity Asset Store)	33
3.1.2	Formati supportati (FBX, OBJ)	34
3.2	Applicazione di animazioni predefinite	35
3.2.1	Animazioni di base: camminata, corsa, salti	36
3.2.2	Librerie di animazioni e compatibilità con i rig	37
3.3	Pipeline di integrazione animazione	37
3.3.1	Esportazione e importazione del modello (FBX → Unity)	38
3.3.2	Animator Controller e gestione degli stati di animazione	40
3.4	Gestione dei modelli animati	41
3.5	Retargeting su umanoidi	41
3.5.1	Concetto di retargeting tra rig simili	42
3.5.2	Metodi automatici semplificati (solo per personaggi umanoidi) . .	42
3.5.3	Limitazioni rispetto a oggetti non organici	43
3.6	Gestione dei modelli inanimati	44
3.7	Preparazione dei modelli	44
3.7.1	Descrizione della mesh	45
3.7.2	Creazione dello scheletro iniziale (da rig umanoide)	46

3.8	Implementazione del rigging automatico in Blender	48
3.8.1	Generazione dello scheletro adattato all'oggetto	49
3.9	Implementazione dello skinning automatico in Blender	55
3.9.1	Trasferimento pesi con algoritmi automatici	56
3.9.2	Collegamento mesh-scheletro e gestione delle deformazioni	59
3.10	Retargeting su oggetti inanimati	59
3.10.1	Trasferimento di animazioni umanoidi agli oggetti	60
3.10.2	Gestione del movimento coerente (root motion o offset spaziali) .	62
4	Esperimenti	65
4.1	Risultati del rigging automatico	65
4.1.1	Confronto visivo del rigging	66
4.1.2	Analisi qualitativa	66
4.2	Risultato dello skinning	67
4.2.1	Confronto visivo delle deformazioni	68
4.2.2	Analisi qualitativi dello skinning	68
4.3	Risultato del retargeting delle animazioni	69
4.3.1	Confronto visivo del retargeting	69
4.3.2	Analisi qualitativa	70
4.3.3	Confronto con altri approcci	71
4.4	Definizione della metrica di errore	73
4.4.1	Motivazioni e obiettivi	73
4.4.2	Definizione della metrica	73
4.4.3	Componenti della metrica	73
4.4.4	Applicazione della metrica ai metodi sperimentati	74
4.4.5	Discussione	75
	Conclusioni e sviluppi futuri	76
	Bibliografia	78

Elenco delle figure

1.1	La schematizzazione della rappresentazione di una mesh	2
1.2	La figura rappresenta la stessa armatura, formata da diverse ossa, in tre posizioni differenti	3
1.3	Traslazione di un oggetto 3D	6
1.4	Matrice di rotazione $R_z(\theta)$ e visualizzazione 3D.	6
1.5	Matrice di rotazione $R_x(\theta)$ e visualizzazione 3D.	7
1.6	Matrice di rotazione $R_y(\theta)$ e visualizzazione 3D.	7
1.7	Scala di un oggetto 3D	8
1.8	Il quaternion <i>quota</i> <i>tail</i> <i>vettore</i> <i>v</i> tramite un angolo θ	10
1.9	Mesh e scheletro umanoide	13
1.10	Assegnazione dei pesi	16
2.1	Logo del software Blender	21
2.2	Logo del software Mixamo	25
2.3	Impostazioni per il download delle animazioni	25
2.4	Logo del software MoveAI	27
2.5	Logo del software Unity3D	29
2.6	Impostazione animazione su Unity	30
3.1	Impostazione animazione su Mixamo	34
3.2	Formato FBX utilizzato anche su AutoCAD	35
3.3	Animazioni base.	36
3.4	Pipeline di integrazione animazione	38
3.5	Proprietà di un oggetto selezionato	39

3.6	Animation Controllor con diverse animazioni	40
3.7	Applicazione animazioni create per un modello	43
3.8	Mesh dei modelli in Blender	46
3.9	Struttura gerarchica	47
3.10	Struttura scheletrica che viene presentato quando se importa uno scheletro su Blender	47
3.11	Fasi per la generazione dello scheletro	50
3.12	Rig Umanoide	55
3.13	Pesi assegnati alla mesh	58
3.14	Vincoli Copy Location su Blender	61
4.1	Risultato del rigging automatico	66
4.2	Risultato dello skinning automatico	68
4.3	Frame rappresentativo del risultato di retargeting dell'animazione sull'ogget- to target ottenuto con UniRig	69
4.4	Rappresentazione qualitativa delle traiettorie del punto articolare intermedio (equivalente al ginocchio) durante il retargeting dell'animazione, proiettate su un piano bidimensionale.	70
4.5	Rappresentazione qualitativa delle traiettorie del punto terminale della gam- ba (equivalente al piede) durante il retargeting dell'animazione.	71
4.6	Confronto qualitativo tra i diversi approcci, evidenziandone punti di forza e limitazioni nel contesto del retargeting di animazioni su oggetti 3D	72

Capitolo 1

Background metodologico

Il presente capitolo ha lo scopo di introdurre il background metodologico su cui si basa il progetto di tesi sviluppato. Nella prima sezione verranno illustrati dei concetti fondamentali che ci aiuteranno a capire nel modo migliore quello che è stato fatto, chiaramente quello illustrato è una sintesi semplificata, la cui descrizione esaustiva e formale (decisamente più complessa) non è rilevante ai fini della comprensione di questa tesi. Nella seconda sezione rappresentiamo il movimento e l'orientamento degli oggetti 3D. Nella terza sezione verrà illustrato come viene controllata una mesh e il modello adottato. Nella quarta sezione come viene trasferito un movimento da un modello ad un altro con diverse struttura e morfologia.

1.1 Mesh 3D e rappresentazioni geometriche

Una mesh è una struttura di dati geometrici che segue la rappresentazione di suddivisioni superficiali da un insieme di poligoni. Meshes sono particolarmente usate in computer graphics, per rappresentare superfici, o nella modellazione, per discretizzare una superficie continua o implicita [Bus20]. Questa mesh può anche essere considerata come la combinazione geometrica e topologica, dove la geometrica fornisce le posizioni di tutti i vertici, e la topologia fornisce le informazioni tra differenti vertice adiacenti.

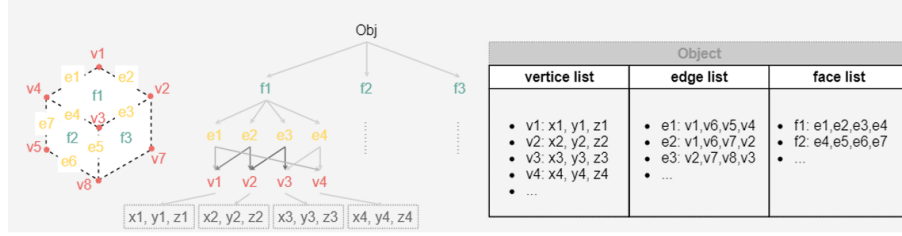


Figura 1.1: La schematizzazione della rappresentazione di una mesh

1.1.1 Vertici, lati, facce e struttura topologica

Una mesh superficiale poligonale ha tre differenti elementi combinati: vertici, bordi e lati. La figura mostra gli elementi 1.1. Matematicamente [NLG23], una mesh superficiale poligonale $\mathcal{M}$ che contiene $\mathcal{V}$ vertici e $\mathcal{F}$ facce è formato da:

$$\mathcal{M} = \{(V, F) \mid V = \{v_i\}_{i=1,2,\dots,V}, F = \{f_i\}_{i=1,2,\dots,F}, f_i \in V^d\}$$

dove $\mathcal{V}$ è l'insieme dei vertici, $\mathcal{F}$ è l'insieme delle facce e d denota il numero di vertici di ogni faccia. Allo stesso modo, un volume di una mesh $\mathcal{M}$ è formato da:

$$\mathcal{M} = \{(V, F, T) \mid T = \{t_i\}_{i=1,2,\dots,t_i}, t_i \in F^k\}$$

dove k rappresenta il numero di facce per ogni voxel t_i .

1.1.2 Coordinate locali e globali, trasformazioni con matrici 4x4

Tutti gli oggetti visualizzati in grafica 3D, si basano sulle posizioni dei loro vertici all'interno dello spazio tridimensionale. Queste coordinate però sono soggette a numerosi cambiamenti durante le animazioni. Come visto, le coordinate rappresentano le posizioni dei vertici di una figura nello spazio 3D. Per visualizzare una scena, un programma di grafica deve conoscere le coordinate di tutti i vertici degli oggetti che devono essere visualizzati. Se l'oggetto viene spostato o ruotato, queste coordinate cambiano di conseguenza anche se la forma dell'oggetto rimane sempre la stessa.

Probabilmente la rappresentazione matriciale è più ottimale in termini di flessibilità, in quanto ti permette di rappresentare anche traslazione, scalature e altre trasforma-

zioni. È, infatti, la rappresentazione che Blender utilizza internamente proprio perché offre la maggior flessibilità. Siccome permette di rappresentare qualsiasi tipo di trasformazione, questa struttura viene solitamente chiamata *Matrice di Trasformazione*. Nel caso di uno spazio a 3 dimensioni, essa a dimensione 4×4 . [Mar19]

Questa sua caratteristica la rende indispensabile per rappresentare articolazioni formate da più ossa, che verranno illustrate di seguito 1.2, poiché rende possibile convertire le coordinate locali di un osso figlio a quelle locali del suo padre, fino ad arrivare alle coordinate globali.

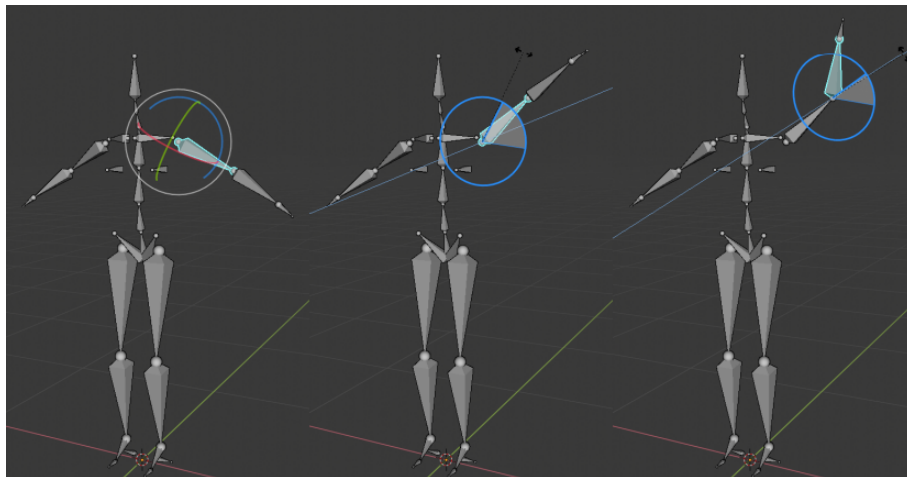


Figura 1.2: La figura rappresenta la stessa armatura, formata da diverse ossa, in tre posizioni differenti

1.1.3 Differenze tra mesh statica e mesh animata

Una mesh statica è una struttura i cui vertici rimangono fissi nel tempo: rappresenta un oggetto immobile o privo di deformazioni, come un modello architettonico o un oggetto di scena. Al contrario, una mesh animata varia la posizione dei suoi vertici nel tempo, secondo una serie di trasformazioni che possono derivare da:

1. Animazione di oggetto (movimento rigido della mesh nel mondo)
2. Rigging e skinning, dove i vertici sono influenzati da uno scheletro intero (armatura) e da pesi di deformazioni

3. Simulazioni fisiche (tessuti, fluidi, soft body), in cui le posizioni vengono aggiornate dinamicamente.

In pratica, una mesh animata non è solo una rappresentazione geometrica, ma un sistema dinamico che reagisce a vincoli, ossa e forze esterne. La gestione efficiente di questa variazione temporale è ciò che rende possibile il movimento realistico nei personaggi e negli oggetti deformabili.

1.2 Trasformazioni geometriche

Nel contesto dell'animazione 3D, ogni elemento visivo - sia esso un personaggio o un oggetto - prende vita attraverso una combinazione di trasformazioni geometriche fondamentali: traslazione, rotazione e scalatura. Tali operazioni sono essenziali per finire le posizione, l'orientamento e la dimensione della mesh nello spazio tridimensionale, e vengono efficacemente rappresentate grazie all'uso di matrici 4x4 in coordinate omogenee. Queste matrici permettono la composizione sequenziale di più trasformazioni, garantendo un controllo elegante e coerente della deformazione e del movimento dei modelli. Parallelamente, l'uso di quaternioni offre un'alternativa potente e matematicamente robusta per la rappresentazione delle rotazioni 3D: evitando problemi come il gimbal-lock, consentendo interpolazioni fluide e una gestione stabile dell'orientamento negli scheletri animati, i quaternioni si affermano come strumento preferito nei motori grafici e nei sistemi di animazione professionali.

Dal punto di vista matematico, tali trasformazioni vengono rappresentate mediante matrici 4x4, che consentono di gestire in modo unificato le operazioni dello spazio omogeneo. La composizione di più trasformazioni viene ottenuta attraverso la moltiplicazione delle rispettive matrici, seguendo in ordine preciso che influisce direttamente sul risultato finale.

Nell'ambito dell'animazione, queste trasformazioni vengono applicate in modo dinamico per determinare la posizione e la deformazione degli oggetti nel tempo. Ad esempio, durante un movimento di un personaggio, le trasformazioni delle singole ossa del rig vengono propagate alla mesh collegata, consentendo la corretta animazione della

superficie. In questo senso, le trasformazioni rappresentano il collegamento diretto tra la rappresentazione matematica e il comportamento visivo del modello tridimensionale.

1.2.1 Traslazione, rotazione e scala 3D: rappresentazione matriciale e composizione

Nel contesto della grafica e dell'animazione tridimensionale, ogni oggetto nello spazio virtuale è definito da un serie di coordinate che ne descrivono la posizione, l'orientamento e le dimensioni. Per modificare queste proprietà si utilizzano le trasformazioni geometriche, operazioni matematiche che permettono di spostare (traslazione), ruotare (rotazione) o ridimensionare (scala) un modello 3D mantenendo la coerenza spaziale della scena.

Tutte queste trasformazioni possono essere rappresentate mediante matrici 4x4, che consentono di combinare un'unica struttura i cambiamenti di posizione, orientamento e scala [Whi]. Questa rappresentazione matriciale è alla base della pipeline grafica moderna (in Blender, Unity e altri ambienti), poiché permette di applicare in modo efficiente e compatto più trasformazioni in sequenza attraverso la composizione matriciale. Di seguito vengono descritte le tre principali trasformazioni elementari che definiscono il comportamento spaziale di una mesh o di un rig:

- **Traslazione 3D**

La traslazione indica lo spostamento dell'oggetto lungo gli assi cartesiani

$$T(d_x, d_y, d_z) = \begin{bmatrix} 1 & 0 & 0 & d_x \\ 0 & 1 & 0 & d_y \\ 0 & 0 & 1 & d_z \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Infatti:

$$T(d_x, d_y, d_z) \cdot \begin{bmatrix} x & y & z & 1 \end{bmatrix}^T = \begin{bmatrix} x + d_x & y + d_y & z + d_z & 1 \end{bmatrix}^T$$

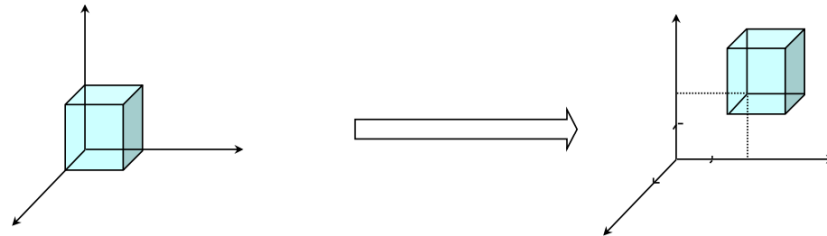
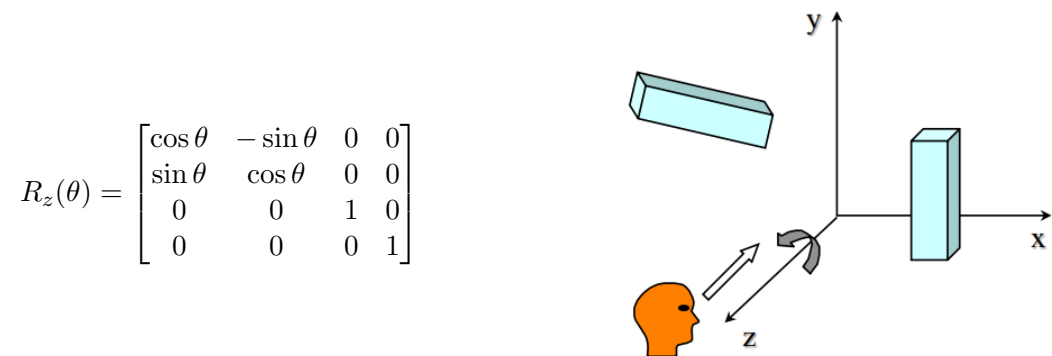


Figura 1.3: Traslazione di un oggetto 3D

• Rotazione 3D

La rotazione definisce l'orientamento nello spazio, in questo caso è necessario definire tre rotazioni base, una attorno a ciascuno dei tre assi x, y e z.



$$R_z(\theta) = \begin{bmatrix} \cos \theta & -\sin \theta & 0 & 0 \\ \sin \theta & \cos \theta & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Figura 1.4: Matrice di rotazione $R_z(\theta)$ e visualizzazione 3D.

$$R_z(\theta) = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos \theta & -\sin \theta & 0 \\ 0 & \sin \theta & \cos \theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

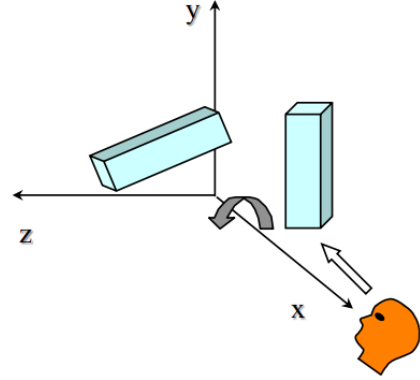


Figura 1.5: Matrice di rotazione $R_x(\theta)$ e visualizzazione 3D.

$$R_z(\theta) = \begin{bmatrix} \cos \theta & 0 & \sin \theta & 0 \\ 0 & 1 & 0 & 0 \\ -\sin \theta & 0 & \cos \theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

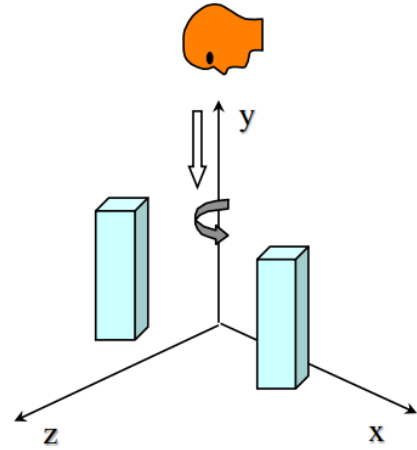


Figura 1.6: Matrice di rotazione $R_y(\theta)$ e visualizzazione 3D.

• Scala 3D

La scala determina l'ingrandimento o la riduzione lungo le diverse direzioni.

$$S(s_x, s_y, s_z) = \begin{bmatrix} s_x & 0 & 0 & 0 \\ 0 & s_y & 0 & 0 \\ 0 & 0 & s_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Infatti:

$$S(s_x, s_y, s_z) \cdot \begin{bmatrix} x & y & z & 1 \end{bmatrix}^T = \begin{bmatrix} x \cdot s_x & y \cdot s_y & z \cdot s_z & 1 \end{bmatrix}^T$$

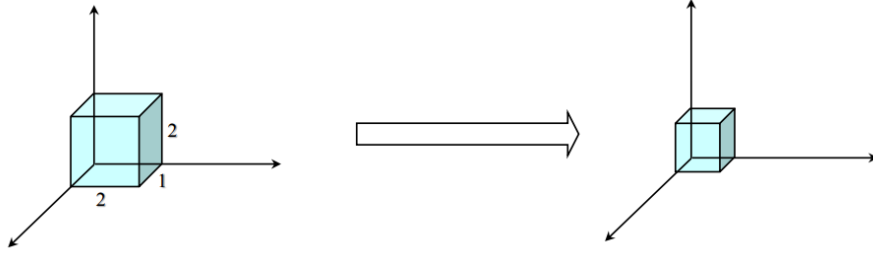


Figura 1.7: Scala di un oggetto 3D

1.2.2 Rotazione con quaternioni: motivazione e vantaggi rispetto agli Euler angles (no gimbal lock)

Nel contesto della grafica tridimensionale, le trasformazioni geometriche costituiscono la base matematica attraverso cui è possibile descrivere la posizione e l'orientamento di un oggetto nello spazio. Storicamente, tali trasformazioni sono state rappresentate mediante matrici di trasformazione, che agiscono come uno stato di riferimento o "base": moltiplicando un vettore o una matrice per la matrice di trasformazione corrispondente, è possibile ottenere la nuova posizione od orientazione dell'oggetto nello spazio tridimensionale. Per la rappresentazione delle rotazioni, un approccio tradizionale è quello basato sugli angoli di Eulero, i quali definiscono l'orientamento di un corpo attraverso una sequenza ordinata di rotazioni elementari attorno agli assi principali del sistema di riferimento (X, Y, Z). Sebbene intuitivi e storicamente molto diffusi, gli angoli di Eulero presentano alcune limitazioni, tra cui in particolare la possibilità di incontrare configurazioni critiche che riducano la stabilità della rotazione.

Per superare tali problematiche, si è progressivamente affermato l'impiego dei **quaternioni**, una formulazione matematica più robusta ed efficiente per la gestione delle rotazioni tridimensionali. I quaternioni consentono di rappresentare in modo compatto un orientamento nello spazio e di calcolare rotazioni continue e interpolazioni fluide tra pose differenti, evitando le ambiguità e le discontinuità tipiche degli angoli di Eulero.

La loro struttura, derivata dai numeri complessi, permette inoltre una maggiore stabilità numerica e una gestione più coerente delle rotazioni cumulative, rendendoli uno strumento essenziale nelle moderne pipeline di animazione e rigging.

- **Angoli di Eulero**

Gli angoli di Eulero rappresentano uno dei metodi più tradizionali per descrivere l'orientamento di un oggetto nello spazio tridimensionale. Essi contengono di esprimere una rotazione come una sequenza ordinata di tre rotazioni elementari attorno agli assi di un sistema di riferimento fisso, generalmente identificati come roll (x), pitch (y) e yaw (z).

Questa rappresentazione è intuitiva, poiché ogni angolo corrisponde a una rotazione attorno ad un asse specifico, permettendo di comprendere facilmente il movimento nello spazio tridimensionale. Gli angoli di Eulero sono stati ampiamente utilizzati in diversi ambiti, tra cui la grafica computazionale, la robotica e l'ingegneria aerospaziale, grazie alla loro semplicità e compatibilità con numerosi software e framework.

Tuttavia, tale rappresentazione soffre di limitazioni strutturali. In particolare, può verificarsi il fenomeno noto come gimbal lock, una condizione in cui due assi di rotazione diventano allineati, riducendo effettivamente i gradi di libertà del sistema. Questa degenerazione comporta instabilità numerica e ambiguità nell'orientamento, soprattutto quando si interagisce con pose molto articolate. Inoltre, l'ordine delle rotazioni (ad esempio, XYZ rispetto a ZYX) influenza direttamente il risultato finale, introducendo ulteriore ambiguità. [Abh24].

Per queste ragioni, nonostante la loro accessibilità concettuale, gli angoli di Eulero sono oggi spesso sostituiti dai quaternioni, i quali garantiscono una rappresentazione continua, priva di singolarità e dunque più adatta ai contesti di animazione e rigging avanzato.

- **Rotazione con Quaternioni**

Nello spazio tridimensionale una rotazione può essere descritta specificando un asse e un angolo di rotazione. Seguendo la convenzione comunemente adottata,

una rotazione con angolo positivo viene eseguita in senso antiorario rispetto alla direzione dell'asse, mentre un angolo negativo genera una rotazione in senso orario.

La rappresentazione tramite asse-angolo, tuttavia, **non è univoca**: la stessa rotazionale può essere ottenuta scalando il vettore asse e modificando l'angolo di un multiplo 2π , come si evidenzia in figura 1.8. Inoltre, la coppia (v, θ) produce la stessa rotazione della coppia $(-v, -\theta)$. Anche se una rotazione può essere espressa tramite una matrice ortogonale 3×3 , questa rappresentazione risulta ridondante e meno efficiente per i calcoli numerici.

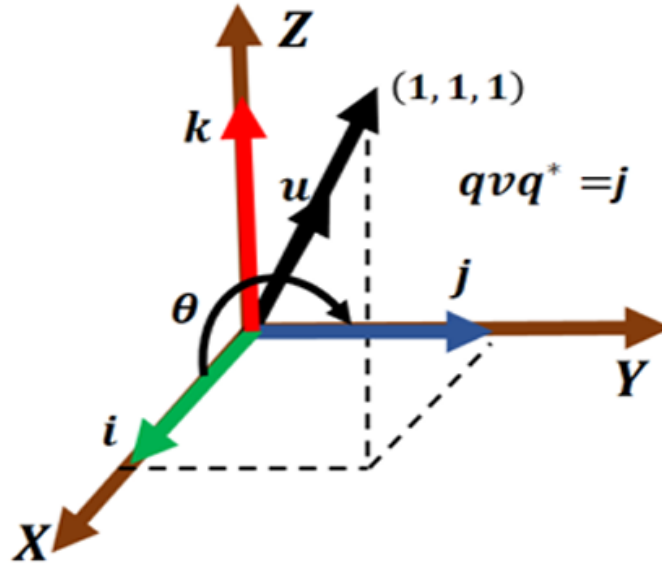


Figura 1.8: Il quaternione qu tramite un angolo θ

Per questo motivo, nel contesto dell'animazione e della grafica 3D si preferisce utilizzare i **quaternioni**, poichè consentono di rappresentare una rotazione tramite quattro soli numeri reali: tre per definire un asse e uno per l'angolo di rotazione.

Un vettore $v \in R^3$ può essere interpretato come un **quaternione puro**, ossia un quaternione con parte reale nulla. Considerando un quaternione unitario nella forma:

$$q = q_0 + q_1i + q_2j + q_3k = q_0 + q,$$

con norma pari a 1, esiste un unico angolo $\theta \in [0, \pi]$ tale che:

$$q_0 = \cos(\theta/2), |q| = \sin(\theta/2)$$

Il quaternioni può quindi essere riscritto come:

$$q = \cos(\theta/2) + u \sin(\theta/2)$$

dove u è un vettore unitario che rappresenta l'asse di rotazione

Definiamo ora un operatore che applica una rotazione a un vettore v mediante il prodotto quaternionico:

$$L_q(v) = qvq^*,$$

dove q^* è il coniugato di q .

Tale operatore presenta due proprietà fondamentali:

1. **Preserva la lunghezza del vettore**, quindi la trasformazione è isometrica:

$$|L_q(v)| = |v|$$

2. **Lascia invariati i vettori paralleli all'asse di rotazione**: se v è proporzionale a u , allora $L_q(v) = v$

Queste proprietà dimostrano che l'operatore L_q applica esattamente una rotazione di ampiezza θ attorno all'asse u [FRGD24]. La compattezza e stabilità di tale rappresentazione la rendono particolarmente adatta alle applicazioni interattive.

L'introduzione dei quaternioni ha permesso di superare queste limitazioni offrendo un modello matematicamente più stabile e continuo. Le principali motivazioni alla base del loro impiego sono le seguenti:

- Assenza di gimbal lock: poiché le rotazioni non dipendono da una sequenza ordinata di assi.
- Interpolazione fluida tra due orientamenti (nota come Spherical Linear Interpolation, o SLERP): fondamentale per animazioni continue.

- Maggiore stabilità numerica, utile in simulazioni e ambienti tridimensionali complessi
- Efficienza computazionale: le operazioni di composizione e inversione risultano più leggere rispetto alle trasformazioni basate su matrici 3×3 o 4×4 .
- Compatibilità con i moderni motori grafici: la maggior parte dei software di animazione 3D e dei motori di rendering utilizza internamente i quaternioni per gestire rotazioni e interpolazioni.

In sintesi, i quaternioni costituiscono un'evoluzione naturale rispetto agli angoli di Eulero, fornendo una rappresentazione più robusta, coerente e adatta alla manipolazione continua degli orientamenti nello spazio tridimensionale.

1.3 Rigging e Skinning

L'animazione di un personaggio digitale si basa sull'associazione tra la sua superficie visibile (mesh) e una struttura articolata interna (scheletro). Il processo che permette questa associazione è noto come rigging e consiste nell'allineare lo scheletro alla mesh e nel definire i pesi di skinning, i quali stabiliscono come il movimento di ciascun giunto influisca sui vertici della superficie. [SH25] Quando si utilizzano dati di motion capture, può essere necessario adattare lo scheletro affinché rispetti le proporzioni della mesh, per poi trasferire (retargeting) il movimento originale al nuovo scheletro. In questo contesto, la configurazione dello scheletro riguarda sia le sue caratteristiche geometriche (come scala e lunghezza delle ossa) sia la sua struttura gerarchica (numero e disposizione dei giunti). Allo stesso modo, la configurazione della mesh comprende sia le sue proporzioni volumetriche sia la topologia dei vertici che la compongono.

1.3.1 Concetto di scheletro e relazione con la mesh

Lo scheletro consiste in un insieme di giunti, come si evidenzia in figura 1.9 (o joint) organizzati in modo gerarchico. Ogni giunto possiede:

- una posizione nella posa di riposo (rest pose)
- una trasformazione locale rispetto al giunto "genitore"
- una trasformazione globale, ottenuta moltiplicando le trasformazioni lungo la gerarchia

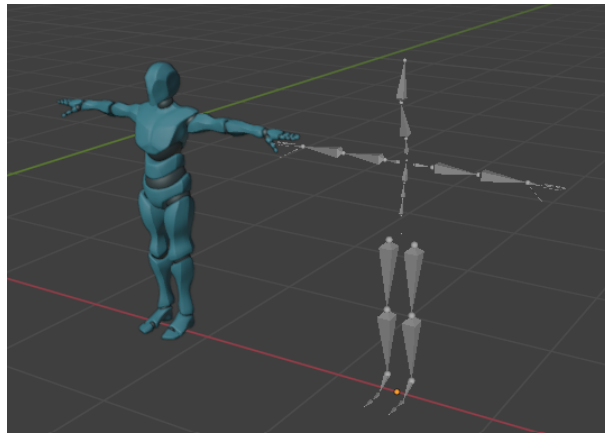


Figura 1.9: Mesh e scheletro umanoide

La gerarchia consente di propagare il movimento: se il bacino ruota, anche la colonna, la testa e le gambe seguono coerentemente. Questo modello prende il nome di rig scheletrico.

La mesh rappresenta invece la superficie visibile del modello ed è formata da un insieme di vertici:

$$V_r \in R^{N_v \times 3}$$

dove N_v è il numero totale di vertici della mesh. Questi vertici descrivono la forma dell'oggetto nella sua posa di riposo.

Per far muovere la mesh insieme allo scheletro si utilizza un processo detto skinning, dove ogni vertice è associato a uno o più giunti tramite pesi di influenza:

$$w_{ij} \geq 0, \sum_{j=1}^{N_j} w_{ij} = 1$$

dove:

- w_{ij} indica quanto il giunto j influenza il vertice i
- N_j è il numero di giunti

1.3.2 Linear Blend Skinning (LBS): formulazione matematica e limiti

Il *Linear Blend Skinning* (LBS) è ampiamente utilizzato nei videogiochi e in molte applicazioni real-time per deformare la superficie di un personaggio in base al movimento di un sottostante scheletro astratto. Consideriamo un modello deformabile rappresentato da una mesh, i cui vertici in coordinate omogenee sono indicate come $\vec{V} = \vec{v}_n \in R^4$.

Il processo di deformazione secondo LBS può essere definito come:

$$\tilde{v}_n = [\sum_b w_{bn} B_b] \bar{v}_n$$

dove l'insieme delle trasformazioni omogenee $B_b \in R^{4 \times 4}$ viene combinato tramite i pesi di skinning w_{bn} . Normalmente questi pesi sono dipinti manualmente da un artista digitale direttamente sulla superficie del modello; tuttavia, esistono anche algoritmi automatici per i casi in cui è richiesta una minore precisione o si desidera ridurre il tempo di produzione.

Ogni matrice di trasformazione B_b è solitamente fattorizzata nel prodotto di due trasformazioni:

$$B_b = \tilde{B}_b \bar{B}_b^{-1}$$

Useremo - per indicare grandezze nello **stato di riposo** (rest pose), e $\sim$ per indicare grandezze nello **stato deformato** (posed). Le trasformazioni $\bar{B}_b^{-1}$ sono definite nella rest pose e rimangono costanti durante tutto il processo di deformazione, mentre le trasformazioni $\tilde{B}_b$ sono calcolate in funzione dei parametri di posa θ , che possiamo astrarre con la funzione:

$$\{\tilde{B}_b\} = pose(\theta)$$

Dato un vertice in rest pose $\bar{v} \in \bar{V}$, la trasformazione:

$$\tilde{v}_B = \tilde{B}_b \bar{B}_v^{-1} \bar{v}$$

esegue due operazioni:

1. **Codifica locale:** $\bar{v}_b = \bar{B}_b^{-1} \bar{v}$, che esprime il punto nel sistema di riferimento locale dell'osso b.
2. **Decodifica globale:** $\tilde{v}_b = \tilde{B}_b \bar{v}_b$, che esprime il punto nel sistema di riferimento globale nella posa corrente.

Infine, rappresentando i pesi $w_n = \{w_n\}^*$ e le trasformazioni $B(\theta) = \{B_b\}$ in forma tensoriale, possiamo sintetizzare lo skinning di un singolo vertice come:

$$\tilde{v}_n = [w_n B(\theta)] \bar{v}_n$$

dove $B(\theta)$ ha dimensioni $B \times 4 \times 4$, e w_n ha dimensioni $1 \times B$. Notiamo anche che i pesi w_n sono normalmente normalizzati affinché siano **positivi** e **sommino a uno**, condizioni necessaria per ottenere una deformazione stabile e priva di artefatti. [Tag20]

Limite dell'LBS. Nonostante la sua semplicità ed efficienza, l'LBS presenta alcune limitazioni note:

1. Collasso del volume (Candy Wrapper Problem)

In presenza di rotazioni ampie, ad esempio la rotazione del braccio o di una gamba, la mesh tende a schiacciarsi invece di mantenere il volume reale.

2. Deformazioni innaturali nelle articolazioni

Le regioni vicino alle giunture (es. gomito, ginocchio) possono assumere forme troppo morbide o eccessivamente elastiche.

3. Difficoltà nella preservazione dei dettagli

LBS non tiene conto della struttura anatomica o materiale: pelle, muscoli o superfici rigide vengono trattati allo stesso modo.

1.3.3 Weight Painting e associazione ossa-vertici

Una volta definito lo scheletro, la mesh deve essere collegata ai giunti in modo che ogni movimento delle ossa si rifletta sul modello. Questo collegamento è realizzato tramite l'assegnazione di pesi ai vertici (skinning weights). Ogni vertice della mesh appartiene a uno o più vertex group, ciascuno associato a un osso. Il peso w_{ij} indica quanto l'osso j influenza il vertice i .

$$0 \leq w_{ij} \leq 1, \sum_j w_{ij} = 1$$

Un peso elevato implica una maggiore deformazione del vertice quando l'osso si muove. La distribuzione dei pesi può essere:

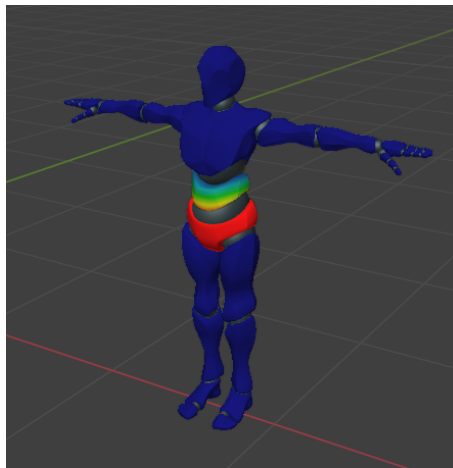


Figura 1.10: Assegnazione dei pesi

- **Automatica**, tramite algoritmi di skinning (es. Automatic Weights in Blender),
- **Manuale**, tramite Weight Painting, dove il modellatore "dipinge" sulla mesh le zone di influenza delle varie ossa.

La corretta assegnazione dei pesi (figura 1.10) è essenziale per ottenere una deformazione naturale nelle zone articolate, come gomiti e ginocchia. Una pesatura scorretta provoca artefatti come collasso del volume, stiramenti e separazioni visibili tra le superficie. Blender visualizza i pesi con una codifica cromatica:

Colore	Significato
Blu	Nessuna influenza (peso ≈ 0)
Verde	Influenza moderata (peso ≈ 0.5)
Rosso	Massima influenza (peso ≈ 1)

Tabella 1.1: Scala cromatica dei pesi nel Weight Painting.

Negli ultimi anni, la ricerca ha introdotto metodi che mirano ad automatizzare ulteriormente questo processo, riducendo la necessità di intervento manuale e migliorando la qualità dello skinning anche su mesh complesse. Tra questi, si distinguono approcci basati su modelli generativi e apprendimento automatico. Ad esempio, alcuni sistemi recenti, come **UniRig** [JPZ25], propongono un generatore unico in grado di produrre scheletri e pesi di skinning coerenti partendo dalla sola geometria della mesh. In modo simile, metodi come **ASMR** [SH25] introducono strategie adattive per prevedere automaticamente la distribuzione dei pesi, sfruttando descrittori geometrici avanzati per individuare le aree di articolazione.

Sebbene tali tecniche non sostituiscano completamente le procedure tradizionali, rappresentano un'evoluzione importante nelle pipeline di rigging moderne, offrendo soluzioni più robuste e generalizzabili e riducendo i tempi di preparazione dei modelli. In questo senso, lo skinning non è più solo un processo manuale, ma si colloca all'interno di un più ampio panorama di metodi automatizzati che mirano a rendere la deformazione della mesh più accurata, stabile e riutilizzabile.

1.4 Retargeting dell'animazione

Il retargeting dell'animazione è il processo con cui un'animazione definita su un personaggio (sorgente) viene trasferita su un altro (target), anche quando i due modelli presentano differenze nella forma, nelle proporzioni o nella struttura dello scheletro. Questo procede richiede:

1. Allineamento degli scheletri (mappatura articolazioni $S \rightarrow T$)
2. Adattamento delle proporzioni (bone scaling)
3. Trasferimento del movimento nel tempo (rotazioni + posizioni articolari)

Storicamente, il retargeting veniva eseguito manualmente tramite Inverse Kinematics e regolazioni osso per osso. Oggi, esistono metodi automatici che combinano:

- analisi strutturate dello scheletro,
- ottimizzazione sui pesi dello scheletro,
- apprendimento automatico.

Tra gli approcci moderni spiccano UniRig e ASMR, che permettono di trasferire animazioni anche su modelli non umanoidi, generalizzando la pipeline oltre il classico personaggio bipede.

1.4.1 Concetto di motion retargeting tra modelli diversi

Il motion retargeting è il processo attraverso il quale un'animazione originariamente associata a un determinato modello o scheletro viene trasferita a un personaggio con proporzioni o struttura differente. La difficoltà principale consiste nel preservare la coerenza del movimento, evitando deformazioni innaturali e garantendo la stabilità del risultato. Con l'evoluzione del machine learning, sono stati studiati approcci che sfruttano reti neurali per apprendere direttamente la mappatura tra scheletri differenti, riducendo la necessità di progettare manualmente vincoli cinematici o modelli di ottimizzazione. Alcuni di questi metodi integrano Forward Kinematics in architetture ricorrenti e introducono vincoli di ciclo per preservare la coerenza del movimento durante la trasformazione. In sintesi, il motion retargeting si è evoluto da metodi basati su ottimizzazione e IK a soluzioni neurali che apprendono direttamente lo spazio delle pose, con l'obiettivo di gestire scheletri eterogenei e movimenti complessi in modo più robusto e automatizzato.

1.4.2 Introduzione ai metodi automatici: UniRig e ASMR come approcci moderni

Negli ultimi anni, il campo del motion retargeting ha visto la diffusione di metodi automatici capaci di trasferire animazioni tra personaggi con proporzioni corporee o strutture scheletriche differenti, riducendo al minimo l'intervento manuale.

Questi approcci sfruttano modelli statistici o reti neurali per apprendere la relazione tra pose, scheletri e deformazioni della mesh, con l'obiettivo di preservare la naturalezza del movimento durante il trasferimento. Tra i metodi recenti, si possono distinguere due linee principali:

Metodi basati su adattamento geometrico (es. UniRig) approcci come **UniRig** mirano a costruire automaticamente uno scheletro coerente con la forma della mesh. L'idea è quella di estrarre informazioni geometriche dal modello (es. simmetria, volumi e articolazioni visive) per posizionare le articolazioni in modo plausibile. Successivamente, vengono calcolati i pesi di skinning e l'animazione può essere trasferita da altri oggetti. Questi metodi sono particolarmente utili quando si lavora con mesh prive di rig o quando si devono rigare personaggi eterogenei in modo uniforme.

Metodi basati su apprendimento (es. ASMR) approcci come ASMR utilizzano reti neurali per apprendere direttamente il processo di retargeting. Il modello è addestrato su numerosi esempi di movimenti eseguiti da scheletri differenti, imparando a mantenere la coerenza del gesto indipendentemente dalla struttura del personaggio. Rispetto ai metodi puramente geometrici, tali tecniche possono adattare automaticamente lo stile del movimento e ridurre distorsioni o artefatti durante la deformazione.

In sintesi UniRig e ASMR rappresentano due direzioni complementari nel motion retargeting moderno:

- l'una focalizzata sulla costruzione automatica dello scheletro e del sistema di skinning,
- l'altra sull'apprendimento della trasformazione del movimento

Entrambi contribuiscono a rendere l'animazione più accessibile, riducendo tempo e complessità nelle pipeline di produzione.

Capitolo 2

Background tecnologico

Il presente capitolo ha lo scopo di presentare gli strumenti software utilizzati nel corso dello svolgimento del progetto di tesi. Non è obiettivo del capitolo fornire una descrizione esaustiva, la quale è facilmente reperibile da altre fonti, ma darne una'introduzione concisa nell'ottica più ampia del progetto realizzato, specificando allo stesso tempo i motivi che hanno portato a scegliere un determinato strumento software piuttosto che un altro.

2.1 Blender

Blender è una suite libera e open-source per la creazione di contenuti 3D. Il software supporta l'intero processo produttivo della grafica tridimensionale: modellazione, rigging, animazione, simulazione, rendering, composizione, motion tracking e anche montaggio video. Blender garantisce un'esperienza coerente su sistemi operativi Linux, macOS e Windows grazie all'utilizzo di OpenGL per l'interfaccia grafica.

Si tratta di uno strumento estremamente versatile, utilizzato in ambiti differenti quali la produzione di animazioni, la creazione di asset per videogiochi, motion graphics, serie televisive, concept art, storyboard, spot pubblicitari e lungometraggi. È impiegato sia da professionisti sia da appassionati e studi di produzione.

Caratteristiche principali:

- Blender è una piattaforma completa per la creazione di contenuti 3D, dotata di tutti gli strumenti fondamentali: modellazione, rendering, animazione e rigging, video editing, effetti visivi, composition, texture e vari tipi di simulazioni fisiche.
- È multiplatforma, con un'interfaccia grafica basata su OpenGL uniforme su tutti i principali sistemi operativi e può essere personalizzato tramite script Python.
- L'architettura interna è progettata per supportare un workflow di produzione rapido ed efficiente.
- Dispone di una comunità molto attiva e distribuita a livello internazionale, con documentazioni, tutorial e risorse disponibili (consultabili, ad esempio, su blender.org/community).
- Può essere installato ed eseguito da qualsiasi directory senza modificare la configurazione del sistema.



Figura 2.1: Logo del software Blender

2.1.1 Modellazione, rigging e animazione

- Modellazione

La modellazione 3D consiste nella creazione della geometria degli oggetti. In Blender questo avviene attraverso la manipolazione di vertici, spigoli (edge) e facce, che insieme formano la mesh. Sono disponibili diversi approcci:

- * Modellazione poligonale (la più comune): si parte da forme primitive e si raffinano progressivamente con strumenti come estrusione, loop cut e subdivide.
- * Sculpting: permette una modellazione più organica e fluida, simile alla scultura tradizionale.
- * Retopologia: processo di semplificazione o ristrutturazione della mesh per renderla più adatta all'animazione.

L'obiettivo della modellazione, nel contesto dell'animazione, è ottenere una geometria con una topologia pulita, cioè organizzata in modo da deformarsi in maniera controllata quando verrà animata.

– Rigging

Il rigging è il processo di creazione di uno scheletro digitale (armatura) che definisce le articolazioni e i punti di movimento dell'oggetto o personaggio. L'armatura è costituita da ossa (bones) collegate da relazioni gerarchiche: quando un osso si muove, i suoi discendenti ereditano la trasformazione. Successivamente si definisce tra scheletro e mesh, tramite lo skinning:

- * Ogni vertice della mesh viene associato a una o più ossa.
- * Le influenze delle ossa sui vertici sono rappresentate da pesi (weight painting).
- * La deformazione della mesh avviene tramite modelli come il Linear Blend Skinning (LBS)

In sintesi, il rigging stabilisce come l'oggetto potrà muoversi, mentre lo skinning stabilisce come la mesh reagisce a tali movimenti.

– Animazione

Una volta configurato il rig, è possibile produrre animazione attraverso vari metodi:

- * KeyFrame Animation: l'animatore definisce manualmente posizioni e pose a istanti temporali specifici, e Blender interpola i movimenti.

- * Inverse Kinematics (IK): si definisce la posizione finale di una parte del corpo (ad esempio il piede) e il sistema calcola automaticamente le articolazioni intermedie.
- * Motion Capture/Retargeting: movimenti registrati da attori reali vengono trasferiti allo scheletro.

Le animazioni sono visualizzate e modificate attraverso strumenti come il **Dope Sheet**, il **Graph Editor** e il **NLA Editor**, che permettono di controllare tempi, curve di interpolazione e combinazione di clip.

2.1.2 Strumenti per scripting in Python (bpy)

Blender integra nativamente un interprete Python e fornisce una API dedicata, denominata bpy, che consente di automatizzare operazioni, creare strumenti personalizzati e interagire con quasi tutti gli elementi presenti nella scena. L'uso di Python rappresenta quindi un modo efficace per estendere a funzionalità del software, ridurre i tempi di lavoro ripetitivi e favorire un workflow riproducibile. L'API bpy permette di:

- accedere e modificare dati della scena (mesh, materiali, armature, animazioni, ecc.);
- eseguire operazioni equivalenti ai comandi della GUI, ma tramite script;
- creare operatori personalizzati, pannelli e addon integrabili nell'interfaccia di Blender;
- automatizzare pipeline complesse come import/export, rigging, baking o generazione di animazioni.

Ad esempio, una mesh può essere letta e modificata tramite codice, i vertex group possono essere manipolati, oppure è possibile controllare direttamente i keyframe di un'animazione. Blender salva inoltre tutte le azioni dell'utente in forma di comandi Python visualizzabili dalla Python Console o dall'Info Log, facilitando l'apprendimento della corrispondenza tra operazioni manuali e comandi programmabili. L'impiego della scripting API è particolarmente utile quando si lavora con

retargeting dell'animazione o generazione procedurale, come nel caso dell'auto-rigging o del trasferimento automatico dei pesi (skinning), poiché consente di operare in modo coerente su più modelli in modo ripetibile e senza intervento manuale.

2.2 Mixamo

Mixamo è una piattaforma online fornita da Adobe che mette a disposizione una vasta libreria di animazioni umanoidi pronte all'uso, progettato per essere applicate direttamente a modelli 3D dotati di rigging scheletrico. La libreria comprende migliaia di movimenti categorizzati per tipologia, come camminate, corse, salti, espressioni corporee, azioni sportive, movimenti stilizzati e animazioni di combattimento. Ogni animazione è creata a partire da dati di motion capture, garantendo una riproduzione realistica della dinamica del movimento umano. Le animazioni sono inoltre parametrizzabili: l'utente può modificare aspetti quali ampiezza dei gesti, velocità, posizione del baricentro o intensità dell'azione direttamente dell'interfaccia del servizio. Questo rende possibile adattare rapidamente un'animazione alle esigenze specifiche del personaggio o della scena, senza necessità di interventi manuali sull'intero ciclo animato. Un aspetto rilevante è la compatibilità con rigging umanoidi standardizzati, come l'armatura "Mixamo Skeleton". Grazie a questo formato, le animazioni possono essere trasferite con facilità a una vasta gamma di modelli 3D, riducendo la necessità di procedure di retargeting complesse. Inoltre, Mixamo offre uno strumento di auto-rigging che consentono di generare automaticamente lo scheletro del personaggio e assegnare i pesi di skinning, partendo da una semplice mesh umanoide.

2.2.1 Esportazione in formato FBX e compatibilità con Blender e Unity

Mixamo consente l'esportazione di modelli riggati e animazioni principalmente nel formato FBX (FilmBox), uno dei formati più diffusi nell'ambito della grafica 3D



Figura 2.2: Logo del software Mixamo

e dei motori di gioco. Il formato FBX conserva tutte le informazioni necessarie per la riproduzione del movimento, inclusi:

- la gerarchia scheletrica (armatura)
- i pesi di skinning (associazione vertice-osso)
- le curve di animazione relative alla rotazione, traslazione e scala dei giunti.

DOWNLOAD SETTINGS

Format	Skin
FBX Binary (.fbx)	With Skin
Frames per Second	Keyframe Reduction
30	none

CANCEL DOWNLOAD

Figura 2.3: Impostazioni per il download delle animazioni

Questa caratteristica rende l'FBX particolarmente adatto all'interscambio tra software differenti, consentendo un flusso di lavoro fluido tra Mixamo → Blender → Unity.

Compatibilità con Blender

All'interno di Blender, l'importazione del file FBX mantiene nella maggior parte dei casi:

- la struttura dello scheletro Mixamo ("mixamorig"),

- le animazioni applicate all’armatura,
- la corretta deformazione della mesh durante il movimento.

Tuttavia, può essere necessario eseguire alcuni passaggi di adattamento, come:

- la normalizzazione delle scale globali,
- la riconversione degli assi di orientamento,
- la rinominazione automatica dei bones per compatibilità con rig controllers (ad esempio, Rigify).

Blender fornisce strumenti per la gestione dell’animazione, tra cui NLA Editor e Dope Sheet, che permettono di combinare, modificare o rifinire le animazioni importate da Mixamo.

Compatibilità con Unity

Nel motore di gioco Unity, i file FBX importati da Mixamo vengono riconosciuti come Humanoid Rig tramite il sistema di retargeting dell’engine. Questo consente:

- l’associazione dei movimenti a personaggi differenti purchè abbiamo un rig umanoide compatibile,
- l’utilizzo delle animazioni all’interno dello Animator Controller,
- la sincronizzazione delle sequenze tramite lo state machine di Unity

Inoltre, Unity consente di:

- miscelare animazioni (blending),
- applicare inverse kinematics in runtime,
- creare animazioni procedurali combinate a quelle keyframed.

Questo rende l’integrazione delle animazioni Mixamo particolarmente efficace nel contesto della produzione di videogiochi, simulazioni interattive e applicazioni VR/AR.

2.3 MoveAI

MoveAI è un'innovativa piattaforma di motion capture basata sull'intelligenza artificiale che converte video ordinari in animazioni 3D di alta qualità, senza bisogno di marcatori, tute o costose configurazioni. Progettata per creatori di ogni livello, MoveAI utilizza la visione artificiale e l'apprendimento automatico per analizzare il movimento umano, fornendo dati di animazione accurati e realistici direttamente da riprese con una o più telecamere. Grazie a strumenti intuitivi come MoveOne a prezzi flessibili basati sui crediti, MoveAI consente a singoli artisti, studi e aziende di creare motion capture di livello professionale a una frazione dei costi tradizionali.



Figura 2.4: Logo del software MoveAI

2.3.1 Generazione automatica di animazioni realistiche da video (MoveAI)

Negli ultimi anni, l'evoluzione dei metodi di motion capture non marked-based ha permesso di ottenere animazioni di qualità cinematografica a partire da semplici video, senza la necessità di tute sensorizzate o sistemi ottici complessi. Tra le tecnologie più avanzate in questo ambito si colloca MoveAI, una piattaforma che utilizza algoritmi di visione artificiale e machine learning per estrarre il movimento umano in modo accurato da sequenze video provenienti anche da più camere commerciali. MoveAI si basa su una pipeline di elaborazione che integra:

- stima della posa 2D in ciascun frame del video;
- ricostruzione tridimensionale della posa mediante triangolazione multi-camera;
- filtraggio temporale per garantire coerenza e fluidità del movimento;
- retargeting automatico del movimento su un modello umanoide virtuale.

Il risultato è un'animazione naturalistica e continua, che rispetta ritmo, velocità, dinamica e stile di movimento originale. A differenza dei sistemi tradizionali di motion capture, che richiedono ambienti controllati e strumentazione costosa, MoveAI consente di registrare l'azione in contesti quotidiani, riducendo drasticamente i costi di produzione e rendendo più accessibile la creazione di animazioni realistiche. Sul piano applicativo, questa tecnologia si rivela particolarmente utile in:

- animazione 3D e produzione cinematografica,
- sviluppo di videogiochi ed esperienze immersive,
- sport e analisi del movimento,
- realtà virtuale e aumentata.

Inoltre, l'output generato può essere esportato direttamente come file FBX, consentendo l'integrazione fluida con software come Blender, Maya, Unity, Unreal Engine, e facilitando il retargeting su personaggi diversi attraverso rig umanoidi standard (come mixamorig o Unity Humanoid Rig).

2.4 Unity 3D

Unity è un motore grafico multiplatforma sviluppato da Unity Technologies che consente lo sviluppo di videogiochi e altri contenuti interattivi, quali visualizzazioni architettoniche o animazioni 3D in tempo reale. L'ambiente di sviluppo Unity gira sia su Microsoft Windows sia su macOS e sia su Linux, e i giochi che produce possono essere eseguiti su Microsoft Windows, macOS, Linux, Xbox 360, PlayStation 3, PlayStation Vita, Wii, iPad, iPhone, Android, Windows Mobile, PlayStation 4, Xbox One, Wii U e Nintendo Switch. Unity consente di utilizzare

sia un editor per lo sviluppo/progettazione di contenuti sia un motore di gioco per l'esecuzione del prodotto finale. Unity è simile a Director, Blender Game Engine, Virtools, Torque Game Builder e GameStudio, è inoltre possibile utilizzare un ambiente grafico integrato come metodo primario di sviluppo.



Figura 2.5: Logo del software Unity3D

2.4.1 Gestione dei modelli animati in Unity

Unity offre un sistema integrato per la gestione di modelli 3D animati, basato sul concetto di Animator, che consente di controllare e riprodurre animazioni in modo dinamico all'interno di una scena interattiva. Una volta importato un modello, Unity riconosce automaticamente la presenza di uno scheletro e associa un rig di animazione configurabile (Figura 2.6) in tre modalità: **Generic**, per personaggi senza struttura umanoide; **Humanoid**, per modelli con anatomia umana standard; e **Legacy**, per animazioni basate su clip non scheletriche. L'utilizzo del rig Humanoid è particolarmente vantaggioso, poiché permette di sfruttare la retargeting map interna di Unity, consentendo così la riutilizzabilità delle animazioni tra personaggi differenti.

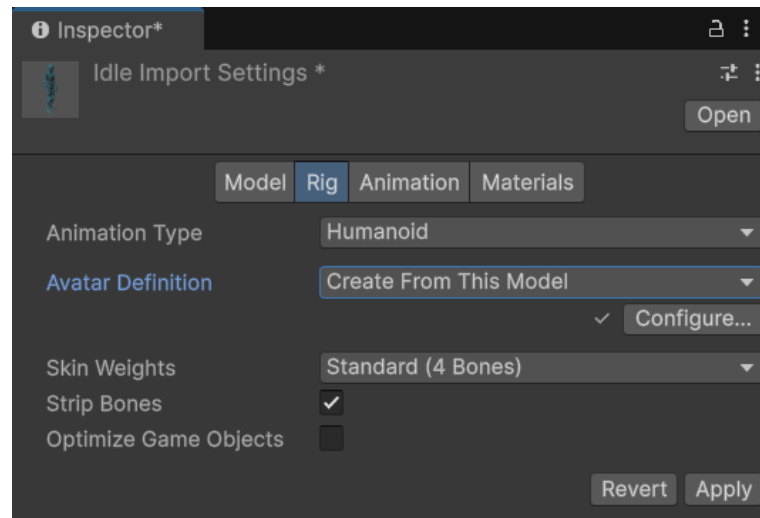


Figura 2.6: Impostazione animazione su Unity

2.4.2 Pipeline di importazione FBX → Unity

Il formato FBX rappresenta lo standard più comune per trasferire modelli animati da software di authoring 3D (come Blender o Maya) verso Unity. Durante l'importazione, Unity applica automaticamente una serie di conversioni, tra cui la normalizzazione delle scale, l'interpretazione dello scheletro e la suddivisione di eventuali clip di animazione contenute nel file. Una volta importato, il modello può essere configurato tramite l'Inspector: è possibile definire il tipo di rig, controllare a coerenza della gerarchia scheletrica e, se necessario, correggere le pose di riferimento (T-Pose/A-Pose). L'animazione può quindi essere associata al personaggio attraverso componenti Animator o Animation.

2.4.3 Sistema di retargeting delle animazioni: Animator Controller

Il cuore del sistema di animazione in Unity è l'Animator Controller, uno strumento che consente di definire come e quando un personaggio esegue determinate animazioni. L'Animator Controller funziona tramite una struttura a **state machine**, dove ogni stato rappresenta una clip di animazione o una combinazione di

esse. Le transizioni tra gli stati possono essere controllate da parametri, variabili che vengono aggiornate in tempo reale dal codice o dall'interazione dell'utente.

Quando si utilizza un rig di tipo Humanoid, Unity applica automaticamente il motion retargeting, ovvero la conversione dei movimenti da una sorgente (ad esempio un'animazione proveniente da Mixamo o MoveAI) al modello in scena. Questo processo preserva l'intenzione del movimento, adattandolo alle proporzioni del personaggio, permettendo così di riutilizzare la stessa animazione su avatar differenti senza ulteriori modifiche.

Capitolo 3

Approccio e metodologia

Nei capitoli precedenti sono stati introdotti i concetti teorici e operativi necessari alla realizzazione del progetto. In particolare nel Capitolo 1 sono stati descritti i fondamenti geometrici e matematici alla base della rappresentazione e manipolazione di oggetti 3D, mostrando come mesh, trasformazioni e sistemi di coordinate permettano di modellare e controllare il movimento nello spazio tridimensionale. Nel capitolo 2, invece, è stata analizzata la pipeline di animazione attraverso tool e software specifici, soffermandosi sul rigging, sullo skinning e sulle tecniche di acquisizione e trasferimento del movimento tramite sistemi di motion capture.

In questo capitolo tali conoscenze vengono applicate nella costruzione del workflow sviluppato per la presente tesi. L'obiettivo è automatizzare il trasferimento di animazioni umanoidi verso modelli non convenzionati, riducendo l'intervento manuale e rendendo il processo più efficiente e scalabile. Verranno quindi illustrate le fasi operative che compongono la pipeline: a partire dall'importazione e test delle animazioni umanoidi, passando per la generazione automatica dello scheletro e l'assegnazione dei pesi alla mesh, fino al retargeting dell'animazione e all'integrazione finale in Unity. Ogni sezione descriverà sia gli strumenti utilizzati sia gli script utilizzati, mettendo in evidenza le scelte progettuali adottate e i risultati ottenuti.

3.1 Importazione dei modelli umanoidi

L'importazione del modello 3D rappresenta il primo passo fondamentale nel processo di animazione di un personaggio umanoide. Affinché un modello possa essere animato correttamente, esso deve essere dotato di una struttura scheletrica interna (rig) che consenta la manipolazione delle articolazioni e la deformazione della mesh. La qualità e la correttezza dei rig influenzano direttamente la fluidità e la naturalezza dei movimenti, rendendo questa fase cruciale per l'intera pipeline di animazione.

In ambito di sviluppo videoludico e applicazioni interattive, i modelli umanoidi vengono generalmente importati da librerie esterne o realizzati tramite software di modellazione 3D. Tuttavia, è necessario assicurarsi che il formato del file sia compatibile con il motore di sviluppo utilizzato: il formato FBX è il più diffuso in quanto permette di importare non solo la geometria del modello, ma anche la rig e, se presenti, le animazioni predefinite.

L'obiettivo di questa sezione è quindi descrivere i principali formati di file utilizzati, le fonti più comuni per l'acquisizione di modelli umanoidi e le impostazioni necessarie per garantire una corretta integrazione del personaggio all'interno dell'ambiente Unity.

3.1.1 Tipologie di modelli (Mixamo, Unity Asset Store)

Piattaforme come Mixamo e Unity Asset Store mettono a disposizione personaggi già riggati, riducendo significativamente i tempi di preparazione e permettendo di concentrarsi sulla fase di animazione.

Mixamo, un servizio offerto da Adobe, mette a disposizione una libreria molto vasta di personaggi 3D completi di scheletro. La peculiarità di Mixamo risiede nel sistema di auto-rigging, che permette di applicare automaticamente uno scheletro umanoide a un modello caricato dall'utente. L'Unity Asset Store, invece, è un marketplace integrato nel motore Unity, che offre modelli 3D sviluppati da artisti e studi professionisti. I modelli presenti nello store variano notevolmente in ter-

mini di complessità, qualità delle texture, dettaglio del rig e presenza o meno di animazioni incluse.

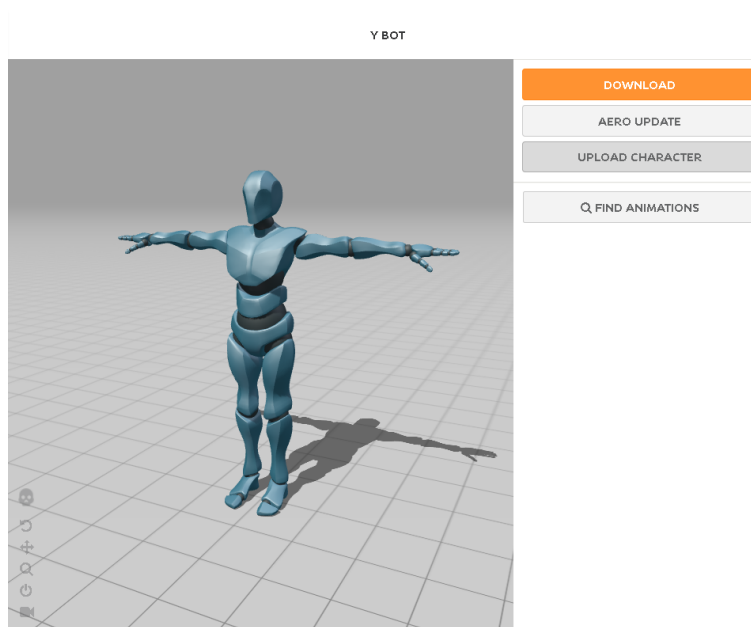


Figura 3.1: Impostazione animazione su Mixamo

3.1.2 Formati supportati (FBX, OBJ)

Nella fase di importazione dei modelli umanoidi in Unity, il formato del file riveste un ruolo fondamentale, poiché determina quali informazioni saranno effettivamente trasferite all'interno del progetto. I due formati più utilizzati in questo contesto sono FBX e OBJ, ciascuno caratterizzato da specifiche funzionalità e limiti.

Il formato FBX (Filmbox) è considerato lo standard nell'ambito dell'animazione 3D e della produzione videoludica. Oltre alla mesh del modello, l'FBX è in grado di contenere al suo interno la struttura scheletrica (rig), le animazioni associate, le informazioni sui materiali e le texture. Grazie a queste caratteristiche, l'FBX permette di trasferire un modello completo direttamente da software di modellazione come Blender, AutoCAD Maya (come in figura 3.2) o 3ds Max a Unity, mantenendo intatta la gerarchia delle ossa e la corretta disposizione delle animazioni. Al contrario, il formato OBJ è molto più semplice e limitato. Esso rappresenta

esclusivamente la geometria della mesh, senza contenere rig né animazioni. Questo lo rende adatto per modelli statici o come base per successivi processi di rigging e animazione, ma non per una importazione diretta di personaggi già animabili.

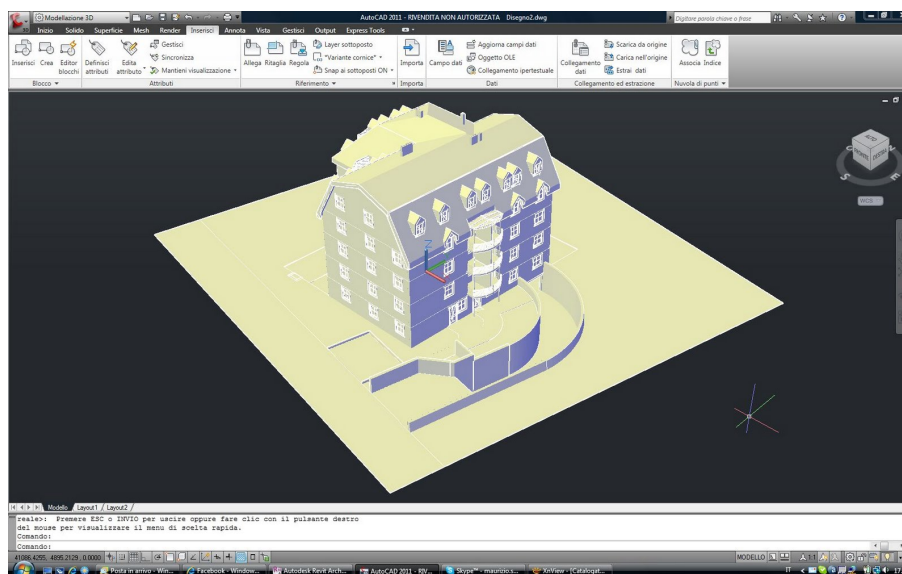


Figura 3.2: Formato FBX utilizzato anche su AutoCAD

In questa fase, l'obiettivo principale è garantire che il modello umanoide sia pronto per ricevere animazioni, senza intervenire ancora su rigging, skinning o retargeting automatico, che saranno approfonditi nei capitoli successivi.

3.2 Applicazione di animazioni predefinite

Una volta importato il modello umanoide nell'ambiente di sviluppo, è possibile procedere con l'applicazione delle animazioni. L'utilizzo di animazioni predefinite consente di ridurre notevolmente i tempi di produzione, evitando la necessità di creare manualmente i movimenti tramite tecniche di keyframing o sistemi di motion capture. Le animazioni predefinite sono generalmente organizzate in librerie, come quelle messe a disposizione da Mixamo, dall'Unity Asset Store o da altri archivi professionali di motion data.

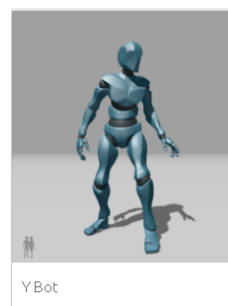
Unity facilita l'integrazione delle animazioni attraverso il sistema di configurazione

del rig Humanoid, che permette di adattare automaticamente le animazioni anche tra modelli differenti. Ciò significa che una stessa animazione, se correttamente configurata, può essere riutilizzata su più personaggi, a condizione che dispongano di una struttura scheletrica compatibile.

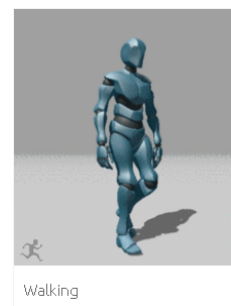
3.2.1 Animazioni di base: camminata, corsa, salti

Le animazione considerate "base" sono quelle che descrivono le azioni fondamentali del movimento umano. Tra le più comuni si trovano:

- **Idle**: posizione di riposo del personaggio
- **Walk cycle**: ciclo completo di camminata.
- **Run cycle**: ciclo di corsa, caratterizzato da maggiore ampiezza e ritmo del movimento.
- **Jump**: animazione di salto, tipicamente suddivisa in tre fasi (preparazione, fase aerea e atterraggio).



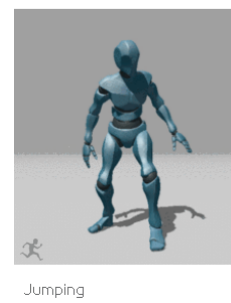
(a) Y-bot



(b) walking



(c) Running



(d) Jumping

Figura 3.3: Animazioni base.

Queste animazioni costituiscono la base per il controllo del personaggio in un ambiente interattivo, come un videogioco o una simulazione. Durante l'importazione delle clip, Unity consente di analizzare e modificare le proprietà delle animazioni, come la ripetizione ciclica (loop), la sincronizzazione dei piedi o la gestione del movimento della radice (root motion), migliorando la fluidità e la naturalezza del risultato finale.

3.2.2 Librerie di animazioni e compatibilità con i rig

Le librerie di animazioni predefinite possono variare per stile, complessità e struttura scheletrica di supporto. Quando si utilizza una libreria, è importante assicurarsi che l'animazione sia compatibile con il rig del modello. Unity supporta questa operazione tramite il sistema di retargeting Humanoid, che consente di adattare le animazioni a scheletri differenti mantenendo la corretta corrispondenza anatomica tra le ossa.

Se il modello e l'animazione sono entrambi configurati come Humanoid, Unity può applicare automaticamente l'animazione anche se proviene da un personaggio con proporzioni diverse. In caso contrario (ad esempio con rig personalizzati o non umani), è necessario intervenire manualmente sulla mappatura delle ossa o utilizzare strumenti esterni di rigging ed editing dell'animazione.

In questo modo, l'adozione della libreria di animazioni predefinite consente di ottenere risultati rapidi e coerenti, permettendo di concentrare gli sforzi creativi sulla logica di interazione e sul comportamento del personaggio piuttosto che sulle fasi dettagliate di creazione del movimento.

3.3 Pipeline di integrazione animazione

L'integrazione delle animazioni all'interno di Unity si basa su una pipeline strutturata che consente di importare correttamente il modello umanoide, configurarne il rig, associare le clip di animazione e gestirne la transizione durante l'esecuzione dell'applicazione. Tale processo assume un ruolo centrale nello sviluppo di personaggi interattivi, perché definisce il modo in cui le animazioni vengono interpretate

e riprodotte dal motore grafico. La pipeline può essere suddivisa in diverse fasi consecutive (come rappresentato in figura 3.11), che vanno dall'esportazione del modello da un software esterno fino alla definizione della logica di controllo dei movimenti attraverso l'**Animator Controller**.



Figura 3.4: Pipeline di integrazione animazione

3.3.1 Esportazione e importazione del modello (FBX → Unity)

Il primo passaggio consiste nell'esportazione del modello e delle relative animazioni da un software di modellazione 3D o da una piattaforma di librerie come Mixamo. Il formato maggiormente utilizzato è FBX, in quanto consente di preservare la struttura gerarchica dello scheletro, le pesature dei vertici e le curve di animazione associate.

Una volta inserito il file FBX all'interno del progetto Unity, il motore provvede alla sua conversione automatica nel formato interno. A questo punto, l'utente può accedere al pannello delle **Import Settings** (Figura 2.6), che consente di definire la tipologia di rig tramite l'impostazione **Humanoid**, garantendo così la compatibilità con il sistema di retargeting del motore. Inoltre, nella sezione Animation, è possibile separare le clip contenute nel file, attivarne il loop, normalizzarne la durata o abilitare l'opzione **Root Motion** (Figura 3.5), che influenzerà la gestione del movimento nello spazio tridimensionale.

Proprietà dell'oggetto selezionato

1. **Controller**: il Animation Controller collegato all'oggetto.
2. **Avatar**: l'Avatar per questo personaggio (se l'Animatore viene usato per animare un personaggio umanoide).

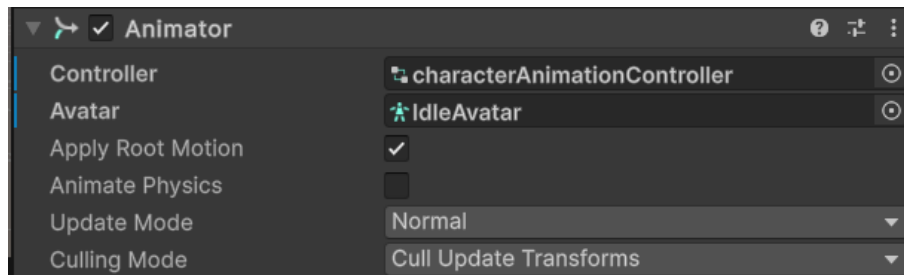


Figura 3.5: Proprietà di un oggetto selezionato

3. **Apply Root Motion:** seleziona se controllare la posizione e la rotazione del personaggio dall'animazione stessa o dallo script.
4. **Update Mode:** questo ti permette di selezionare quando l'Animator aggiorna e quale scala temporale deve utilizzare
 - a) **Normale:** L'Animator viene aggiornato in sincronia con la chiamata Update, e la velocità dell'animatore corrisponde a quella attuale. Se la scala temporale è rallentata, le animazioni rallenteranno per adattarsi.
 - b) **Animate Physics:** L'animatore viene aggiornato in sincronia con la chiamata FixedUpdate (cioè in perfetta sincronia con il sistema fisico). Dovresti usare questa modalità se stai animando il movimento di oggetti con interazioni fisiche, come i personaggi che possono spingere oggetti rigidbody.
 - c) **Unscaled Time:** L'animatore viene aggiornato in sincronia con la chiamata Update, ma la velocità dell'animatore ignora la scala temporale attuale e anima comunque al 100% della velocità. Questo è utile per animare un sistema GUI a velocità normale usando scale temporali modificate per effetti speciali o per mettere in pausa il gameplay.
5. **Culling Mode:** la modalità culling puoi scegliere per le animazioni.
 - a) **Always animate:** animare sempre, non fare culling anche fuori campo.
 - b) **Cull Update Transforms:** Retarget, IK e scrittura delle trasformazioni sono disabilitate quando i renderer non sono visibili.

- c) **Cull Completely**: l'animazione è completamente disabilitata quando i renderer non sono visibili.

3.3.2 Animator Controller e gestione degli stati di animazione

Una volta importate le animazioni, è necessario definire le modalità con cui il personaggio passerà da un movimento all'altro durante l'esecuzione. Unity utilizza a tal fine l'**Animator Controller** (Figura 3.6), una macchina a stati finiti che rappresenta le diverse animazioni come nodi collegati da transizioni. Ogni transizione può essere controllata tramite parametri (booleani, interi, float o trigger), modificabili a runtime tramite script o input dell'utente.

Questo approccio consente di gestire in modo modulare e scalabile sequenze complesse di movimenti, come la transizione fluida tra camminata e corsa o la sincronizzazione tra salto e atterraggio. Inoltre, Unity mette a disposizione strumenti avanzati come i **Blend Trees**, che permettono di interpolare più animazioni in funzione di un parametro continuo (ad esempio la velocità), garantendo una maggiore naturalezza nell'esecuzione delle animazioni di locomozione.

Un'architettura dell'Animator Controller ben progettata consente quindi una maggiore flessibilità, favorisce il riutilizzo delle animazioni e semplifica l'eventuale estensione futura del sistema di controllo del personaggio.

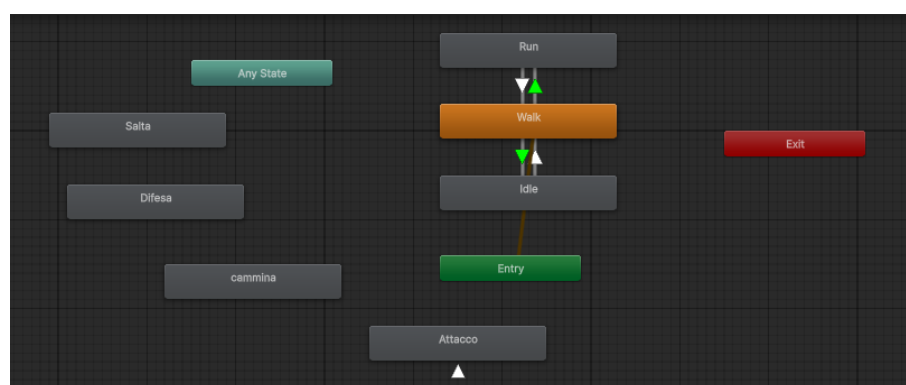


Figura 3.6: Animation Controller con diverse animazioni

3.4 Gestione dei modelli animati

Nel progetto sviluppato, la gestione del modello umanoide risulta semplificata dal fatto che il personaggio utilizzato era già dotato di rig completo e di corretta associazione tra mesh e scheletro. Pertanto, non è stato necessario intervenire sul processo di *skinning* o sulla definizione dei pesi dei vertici, operazioni che normalmente richiederebbero l'utilizzo del software di rigging come Blender o Maya. Unity ha riconosciuto automaticamente la struttura scheletrica del modello e ha provveduto a configurarla secondo il sistema **Humanoid**, permettendo così di applicare animazioni predefinite senza ulteriori modifiche.

La riproduzione delle animazioni è stata gestita tramite l'**Animator Controller**, che ha permesso di definire e controllare le transizioni tra i diversi stati animati, come ad esempio il passaggio da idle a camminata o corsa. Poiché il modello era già predisposto per l'utilizzo delle animazioni umane standard, l'applicazione delle clip è avvenuta in modo diretto, senza la necessità di operazioni di retargeting manuale o adattamento delle pose.

Questa condizione ha semplificato la pipeline di animazione, rendendo il processo più rapido ed efficiente. Rispetto a modelli non organici o non riggati, dove sarebbe necessario costruire uno scheletro artificiale e definire le influenze delle ossa sulla superficie della mesh, nel caso degli umanoidi la struttura preesistente consente un'applicazione immediata e riutilizzabile delle animazioni.

In sintesi, l'utilizzo di un modello già riggato e compatibile con il sistema Humanoid di Unity ha permesso di concentrarsi esclusivamente sulla gestione dell'Animator Controller e delle animazioni, semplificando notevolmente il flusso di lavoro rispetto a oggetti non animati o con rig personalizzato.

3.5 Retargeting su umanoidi

Il retargeting è una procedura che consente di applicare la stessa animazione a modelli diversi, purché condividano una struttura scheletrica compatibile. Nel caso dei personaggi umanoidi, Unity dispone di un sistema di retargeting integrato

basato sullo standard **Humanoid Rig**, che permette di trasferire animazioni tra modelli differenti mantenendo la coerenza dei movimenti. Questo processo risulta particolarmente utile quando si utilizzano librerie di animazioni esterne, come quelle fornite da Mixamo, oppure quando si integrano personaggi provenienti da diverse fonti di modellazione 3D (tipo MoveAI).

3.5.1 Concetto di retargeting tra rig simili

Il sistema Humanoid di Unity definisce una mappatura standard delle principali articolazioni del corpo (test, colonna vertebrale, braccia, gambe, mani). Quando il modello viene importato e configurato come Humanoid, Unity riconosce automaticamente l'equivalente dello scheletro intero e consente di utilizzare animazioni create per altri personaggi umanoidi. In termini pratici, ciò significa che un'animazione di camminata può essere applicata a qualsiasi modello umanoide senza la necessità di ricreare manualmente il movimento.

3.5.2 Metodi automatici semplificati (solo per personaggi umanoidi)

Unity gestisce il retargeting in modo quasi completamente automatico. È sufficiente:

1. Importare il modello come **Humanoid Rig**
2. Associare una clip di animazione proveniente da un'altra sorgente
3. Applicarla tramite l'**Animator Controller**

Grazie a questa funzione, è possibile riutilizzare un'ampia gamma di animazioni già presenti nelle librerie, riducendo significativamente il tempo richiesto per la produzione di movimenti realistici.

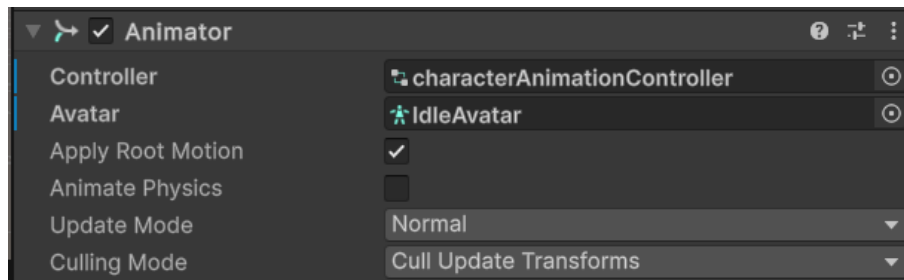


Figura 3.7: Applicazione animazioni create per un modello

3.5.3 Limitazioni rispetto a oggetti non organici

Il retargeting automatico presente nei software moderni (es. Blender, Unity, MotionBlender) si basa quasi sempre su un modello **umanoide standardizzato**, caratterizzato da una struttura scheletrica prevedibile (colonna vertebrale, braccia, gambe, testa) e da vincoli cinematici tipici dell'anatomia umana.

Quando l'oggetto da animare **non possiede caratteristiche antropomorfe**, questa pipeline non risulta applicabile o perde grande parte della sua efficacia. Oltre ai casi già citati (creature non umanoidi, oggetti meccanici, strutture articolate atipiche), esistono varie categorie problematiche:

Perché il retargeting automatico fallisce:

1. **Assenza di una corrispondenza diretta tra giunti.** I sistemi umanoide prevedono un mapping "uno-a-uno" tra ossa equivalenti (Hips → Hips, Spine → Spine, ecc.).
2. **Topologia scheletrica non compatibile.** Creature multipedali, insetti, robot od oggetti inventati presentano:
 - più arti o meno arti
 - articolazioni non lineari (segmenti meccanici, giunti cilindrici, pistoni, tentacoli)
 - strutture non gerarchiche
3. **Range di movimento non umano.** Gli algoritmi umanoidi assumono vincoli cinematici:

- cavallo pelvico
- limitazioni di abduzione/adduzione
- articolazioni sferiche/hinge Oggetti artificiali possono violare tutti questi limiti (rotazione 360 gradi, assi multipli, traslazioni)

4. **Root motion non universale.** Il movimento radice negli umanoidi è centrato su Hips/Pelvis. In un robot o in una creatura aliena, la "root" può essere ovunque.

3.6 Gestione dei modelli inanimati

Nel caso degli oggetti inanimati, la gestione dell'animazione ha richiesto un flusso di lavoro più articolato rispetto a quello adottato per il modello umanoide, in quanto i modelli utilizzati non erano originariamente dotati di uno scheletro né di un sistema di skinning. Di conseguenza, è stato necessario procedere con l'implementazione automatica del rigging e dello skinning, al fine di rendere gli oggetti animabili all'interno dell'ambiente di sviluppo.

Il processo ha previsto la generazione di una struttura scheletrica artificiale, composta da un insieme di ossa funzionali al tipo di movimento desiderato. Successivamente, è stata applicata una procedura di skinning automatico, che ha permesso di associare la mesh alle ossa generate, distribuendo i pesi dei vertici in modo coerente senza interventi manuali. Questa fase, sebbene automatizzata, risulta fondamentale per garantire una deformazione corretta dell'oggetto durante l'animazione.

3.7 Preparazione dei modelli

La preparazione dei modelli costituisce un passaggio fondamentale nella pipeline di animazione 3D, poiché determina sia la struttura geometrica degli oggetti sia la configurazione dello scheletro necessario per il loro movimento. In questo capitolo, si descrive il processo seguito per la creazione di modelli modulari e per la

generazione automatica di un rigging iniziale utilizzando **Blender** come ambiente di lavoro principale.

L'obiettivo di questa fase è duplice: da un lato, garantire che le mesh siano organizzate e pronte per essere animate; dall'altro, creare uno scheletro coerente e funzionale, in grado di supportare deformazioni realistiche e di integrarsi con animazioni predefinite. Per facilitare questo processo, sono stati impiegati riferimenti a scheletri umanoidi standard, adattati alle specificità dei modelli utilizzati.

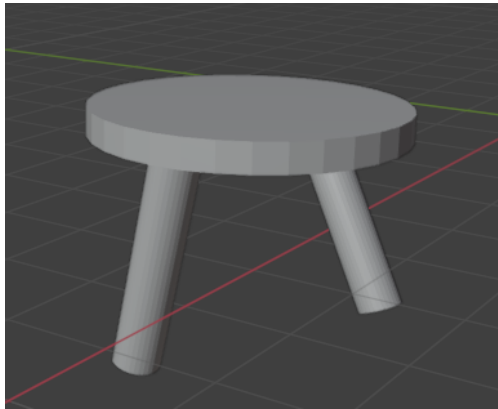
3.7.1 Descrizione della mesh

La fase di definizione delle mesh costituisce il primo passo nella preparazione dei modelli per l'animazione in Blender. In questo lavoro, i modelli sono stati realizzati in forma modulare, al fine di consentire una gestione semplice e flessibile delle diverse componenti durante la fase di rigging, skinning e animazione.

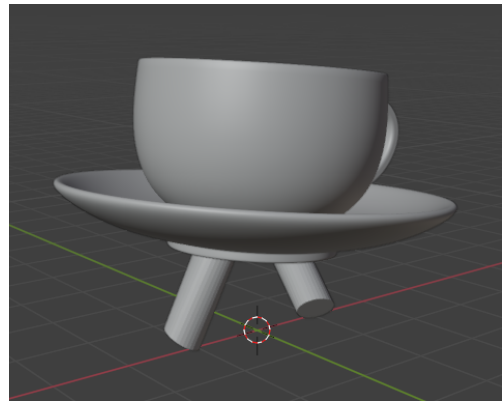
I modelli principali utilizzati sono i seguenti:

- **Piatto con gambe:** rappresenta una struttura base costituita da una mesh piana (piatto) che funge da corpo principale, integrata con due gambe posizionate simmetricamente per consentire il movimento verticale e traslatorio. Questa configurazione permette di testare deformazioni e animazioni di base come camminata o rotazioni.
- **Piattino con tazzina e gambe:** questa combinazione rappresenta una forma più complessa, in cui la tazzina è posizionata sopra il piattino, entrambi supportati da due gambe. Tale struttura offre un maggiore livello di articolazione e permette di sperimentare il comportamento delle mesh durante animazioni più complesse, simulando un personaggio modulare con elementi sovrapposti.

Ogni mesh è stata importata in Blender come oggetto separato, con una propria origine e pivot point, per facilitare l'allineamento con lo scheletro e garantire una corretta deformazione durante l'animazione. L'approccio modulare consente inoltre di sostituire o modificare singoli componenti senza compromettere l'intera struttura del modello.



(a) Piatto con gambe



(b) Piattino con tazzina e gambe

Figura 3.8: Mesh dei modelli in Blender

3.7.2 Creazione dello scheletro iniziale (da rig umanoide)

La definizione dello scheletro iniziale rappresenta il passaggio fondamentale per consentire l'animazione di un modello. Lo **scheletro**, o *rig*, è costituito da una struttura gerarchica (Figura 3.10) di ossa che permette la deformazione controllata delle mesh durante il movimento. La scelta di utilizzare un **rig umanoide come riferimento**, come si evidenzia in figura 3.13, deriva dalla necessità di disporre di una struttura articolare coerente, facilmente adattabile a modelli anche non antropomorfi.

Un rig umanoide standard generalmente composto da elementi ricorrenti, quali:

- una colonna strutturale centrale (spina dorsale);
- due arti inferiori con articolazione dell'anca, del ginocchio e della caviglia;
- due arti superiori con articolazione della spalla, del gomito e del polso;
- un nodo centrale di orientamento (pelvis o root).

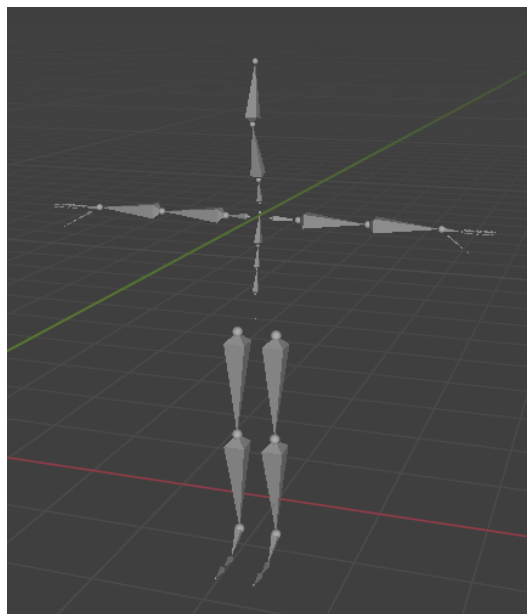


Figura 3.9: Struttura gerarchica

Sebbene i modelli trattati in questo lavoro non riproducano figure umane, la **logica articolare umanoide** risulta comunque utile. Le "gambe" dei modelli, infatti, devono essere in grado di simulare movimenti come passo, flessione o rotazione, che trovano un riferimento diretto nelle articolazioni del corpo umano.

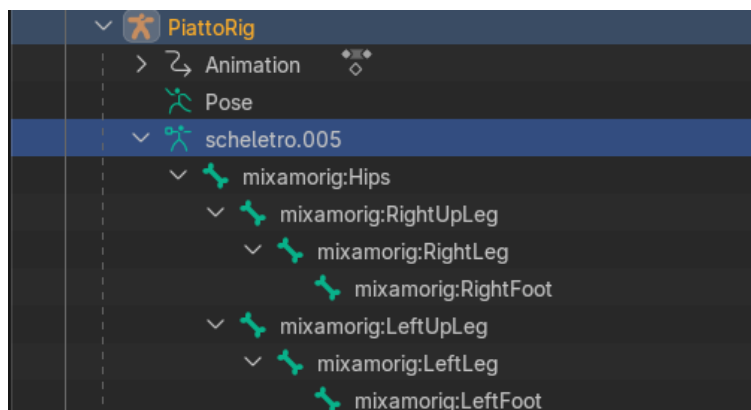


Figura 3.10: Struttura scheletrica che viene presentato quando se importa uno scheletro su Blender

L'adozione di una **struttura scheletrica standardizzata** offre inoltre due van-

taggi rilevanti:

1. **Compatibilità con librerie di animazioni esistenti:**

Un rig umanoide consente di utilizzare animazioni già pronte, riducendo la necessità di creare ogni movimento manualmente.

2. **Facilità di adattamento (retargeting):**

la corrispondenza tra ossa standard rende possibile trasferire animazioni da un modello all'altro, anche quando le proporzioni o le forme sono differenti.

In sintesi, la creazione dello scheletro iniziale si basa sulla definizione di una struttura ossea coerente, gerarchica e compatibile con sistemi di animazione già esistenti. Nel punto successivo verrà descritta l'applicazione pratica di questo processo.

3.8 Implementazione del rigging automatico in Blender

La generazione automatica di scheletri tridimensionali rappresenta una delle problematiche centrali nell'ambito dell'animazione digitale. Essa consiste nel definire una struttura articolata che consenta di deformare in modo realistico una mesh 3D durante il movimento. La predizione di una struttura scheletrica valida a partire da una mesh tridimensionale è un compito complesso, in quanto richiede di catturare sia la geometria sia la topologia dell'oggetto, mantenendo relazioni coerenti tra le diverse articolazioni.

I metodi tradizionali di rigging automatico si basano spesso su modelli o schemi predefiniti, progettati per una gamma limitata di morfologie. Tuttavia, tali approcci risultano poco flessibili e mostrano difficoltà nel gestire modelli con strutture non convenzionali o topologie molto diverse da quelle umane. Per superare questi limiti, le ricerche più recenti hanno proposto approcci di tipo **sequenziale** o **autoregressivo**, in cui lo scheletro viene generato passo dopo passo, prevedendo la posizione di ogni nuovo giunto in relazione a quelli già creati. In questo modo, la costruzione della struttura ossea tiene conto della dipendenza gerarchica

tra le articolazioni e della forma complessiva della mesh, garantendo una maggiore coerenza del risultato finale.

Un ulteriore problema affrontato da tali metodi riguarda la rappresentazione computazionale dello scheletro. Rappresentare una struttura gerarchica in un formato sequenziale non è banale, poiché occorre descrivere sia le coordinate spaziali delle ossa sia le relazioni di parentela tra esse. Soluzioni semplicistiche, che ordinano le ossa secondo percorsi predefiniti, possono introdurre ridondanza e inefficienze, oltre a rendere difficile il rispetto dei vincoli strutturali.

Per gestire in modo efficace queste relazioni, alcuni approcci moderni propongono tecniche di **codifica strutturale** che discretizzano le informazioni spaziali e gerarchiche dello scheletro, in modo da preservarne la coerenza tipologica. Tali strategie consentono di ottenere scheletri più flessibili, adattabili e correttamente articolati, anche nel caso di modelli non organici o con forme atipiche.

I principi teorici alla base di queste metodologie hanno fornito la base concettuale per lo sviluppo di procedure automatizzate di rigging, come quelle realizzata in Blender mediante script Python, descritta nella sezione successiva.

3.8.1 Generazione dello scheletro adattato all'oggetto

A partire dai principi teorici descritti nella sezione precedente, è stata sviluppata una procedura di **rigging automatico** all'interno di *Blender*, con l'obiettivo di generare in modo semi-automatizzato lo scheletro di oggetti tridimensionali non convenzionali, come un piatto o una tazzina dotati di gambe. Il processo è stato realizzato attraverso uno script *Python* personalizzato, concepito per adattare un rig umanoide esistente alla morfologia del nuovo modello, riducendo l'intervento manuale richiesto nel rigging tradizionale.

Il metodo di implementazione prevede diverse fasi operative:

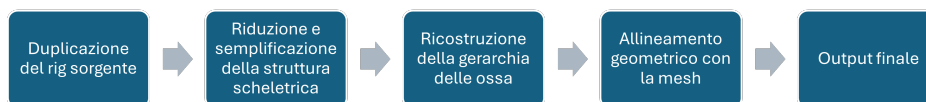


Figura 3.11: Fasi per la generazione dello scheletro

Queste fase operative presentate saranno spiegate successivamente.

1. Duplicazione del rig sorgente

Il punto di partenza è costituito da un rig umanoide di riferimento, già presente nella scena di Blender. Lo script ne crea una copia indipendente, mantenendone la struttura di base e i vincoli di orientamento. Questa operazione consente di preservare l'integrità del modello sorgente e di riutilizzare la sua gerarchia ossea come struttura iniziale per il nuovo oggetto.

Algorithm 1: DuplicateSourceRig

Input: sourceRig, newRigName

// sourceRig: nome dell'armatura sorgente

// newRigName: nome della nuova armatura

src ← getObjectFromScene(sourceRig);

// Recupera l'oggetto armatura sorgente dalla scena

if *src does not exist* **then**

raise Error("Rig sorgente non trovato");

// Interrompe l'esecuzione se l'armatura non è presente

plateRig ← copyObject(src);

// Crea una copia dell'oggetto armatura

plateRig.data ← copyRigData(src.data);

// Duplica i dati dell'armatura per evitare riferimenti
 condivisi

plateRig.name ← newRigName;

// Assegna il nuovo nome all'armatura duplicata

linkObjectToScene(plateRig);

// Collega la nuova armatura alla collezione attiva della
 scena

setActiveObject(plateRig);

// Imposta la nuova armatura come oggetto attivo

2. Riduzione e semplificazione della struttura scheletrica

Poiché l'obiettivo è quello di animare un oggetto con caratteristiche morfologiche ridotte rispetto a un corpo umano, lo script elimina automaticamente le ossa non necessarie (braccia, colonna vertebrale, mani, ecc.), mantenendo soltanto la parte inferiore dei rig - anca, gamba e piedi - sufficiente per conferire al modello la possibilità di movimento e stabilità.

Algorithm 2: KeepLowerBodyBones

Input: plateRig

```
// Imposta l'armatura in modalità di modifica
SetMode(plateRig, EDIT);
// Recupera l'insieme delle ossa in modalità edit
editBones ← plateRig.getEditBones();
// Definisce l'elenco delle ossa da mantenere
keepBones ← ∅;
addBone(keepBones, Hips);
addBone(keepBones, LeftUpLeg);
addBone(keepBones, LeftLeg);
addBone(keepBones, LeftFoot);
addBone(keepBones, RightUpLeg);
addBone(keepBones, RightLeg);
addBone(keepBones, RightFoot);
```

3. Ricostruzione della gerarchia delle ossa

Dopo la rimozione delle parti superflue, vengono ristabilite le connessioni parent-child tra le ossa rimanenti, preservando la sequenza logica "Hips → UpLeg → Leg → Foot". In questo modo il rig conserva una struttura coerente con il sistema di animazione di Blender e risulta pienamente compatibile con il motore di animazione.

Algorithm 3: RebuildBoneHierarchy

Input: editBones

```
// Ricostruisce le relazioni parent-child
// per garantire una corretta propagazione
// delle trasformazioni
```

foreach *side* $\in \{Left, Right\}$ **do**

```
    upLeg  $\leftarrow$  editBones.get(side + "UpLeg");
    // Recupera l'osso della coscia
    leg  $\leftarrow$  editBones.get(side + "Leg");
    // Recupera l'osso della gamba
    foot  $\leftarrow$  editBones.get(side + "Foot");
    // Recupera l'osso del piede
    hips  $\leftarrow$  editBones.get("Hips");
    // Recupera l'osso dei fianchi
    if upLeg exists and hips exists then
        | setParent(upLeg, hips);
        | // Collega la coscia ai fianchi
    if leg exists and upLeg exists then
        | setParent(leg, upLeg);
        | // Collega la gamba alla coscia
    if foot exists and leg exists then
        | setParent(foot, leg);
        | // Collega il piede alla gamba
```

4. Allineamento geometrico con la mesh

Una volta semplificato e ricostruito, lo scheletro viene riposizionato in modo automatico in relazione alla mesh dell'oggetto. Lo script calcola il *bounding box* della geometria e ne individua il centro, in modo da posizionare correttamente le articolazioni principali (anca e gambe) in base alla scala e alla simmetria dell'oggetto. Questa operazione consente di ottenere un rig proporzionato o perfettamente adattato al modello 3D.

Algorithm 4: AlignRigToPlateCenter

```

Input: plate, editBones

// Calcola i vertici del bounding box del piatto
bbox ← GetBoundingBoxVertices(plate);
// Calcola il centro del bounding box
center ← ComputeCenter(bbox);
// Calcola i limiti verticali del bounding box
minZ ← GetMinZ(bbox);
maxZ ← GetMaxZ(bbox);
// Calcola i limiti laterali del bounding box
minX ← GetMinX(bbox);
maxX ← GetMaxX(bbox);
// Recupera l'osso dei fianchi
hips ← editBones.get(Hips);
// Riposiziona testa e coda dell'osso dei fianchi
SetHead(hips, center);
SetTail(hips, center + (0, 0, (maxZ - minZ) × 0.2));
// Riposiziona le ossa delle cosce
foreach (side, xOffset) ∈ {(Left, minX × 0.7), (Right, maxX ×
0.7)} do
    upLeg ← editBones.get(side + "UpLeg");
    if upLeg exists then
        SetHead(upLeg, center + (xOffset, 0, 0));

```

5. Output finale

Al termine dell'esecuzione dello script Python, viene generato uno **scheletro adattato alla geometria dell'oggetto**, con una gerarchia coerente e correttamente allineata alla mesh. In questa fase, tuttavia, il rig non è ancora completamente pronto per l'animazione, poiché **non è stato eseguito lo skinning**, ovvero il processo di associazione dei vertici della mesh alle ossa con pesi di influenza.

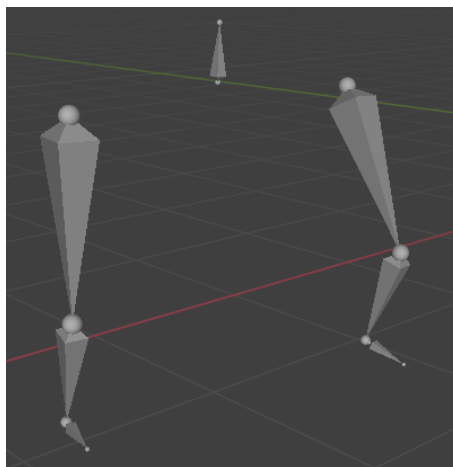


Figura 3.12: Rig Umanoide

L'approccio adottato rappresenta una versione semplificata dei metodi di rigging automatico più avanzati, traducendo in una procedura pratica alcuni dei principi teorici descritti nel capitolo precedente. Pur non implementando algoritmi di previsioni basati su modelli neurali o processi autoregressivi, lo script sviluppato in Blender consente di ottenere una **configurazione scheletrica adattativa**, riducendo in modo significativo i tempi di preparazione del rig e permettendo l'applicazione di animazione predefinite anche a modelli non organici.

3.9 Implementazione dello skinning automatico in Blender

Dopo la generazione automatica dello scheletro, la fase successiva del processo consiste nell'associazione tra la mesh nell'oggetto e la struttura scheletrica, ovvero lo **skinning**. Lo skinning rappresenta un passaggio fondamentale nel rigging, poichè determina in che modo i vertici della mesh si deformano in risposta ai movimenti delle ossa. L'obiettivo di questa fase è garantire una **deformazione realistica e coerente** dell'oggetto durante l'animazione, riducendo al minimo le distorsioni indesiderate.

Nel contesto di questo progetto, lo skinning è stato implementato in *Blender* attraverso un **procedimento automatico**, con l'obiettivo di trasferire i pesi di influenza da un modello sorgente (un rig umanoide già animato) a un oggetto non organico, come un piatto o una tazzina dotati di gambe.

3.9.1 Trasferimento pesi con algoritmi automatici

Nel contesto della generazione automatica di rig, un aspetto fondamentale riguarda la previsione dei pesi di skinning, ovvero i coefficienti che definiscono l'influenza di ciascun osso sui vertici della mesh. La determinazione di questi pesi è un problema di natura complessa, poiché implica la definizione di una matrice ad alta dimensionalità, in cui ogni riga corrisponde a un vertice della mesh e ogni colonna a un osso del rig. Nei modelli tridimensionali di alta risoluzione, tale matrice può raggiungere dimensioni considerevoli, rendendo il processo computazionalmente oneroso.

Le ricerche più recenti in ambito accademico hanno proposto approcci data-driven e basati su reti neurali autoregressive per stimare automaticamente tali pesi, integrando anche parametri aggiuntivi legati alle proprietà fisiche delle articolazioni (come rigidità o gravità). Questi metodi sfruttano meccanismi di cross-attention per modellare in modo efficiente la relazione tra la topologia dello scheletro e la geometria della mesh, migliorando la coerenza del movimento e la qualità della deformazione.

Nel caso di questo progetto, pur non adottando una soluzione di apprendimento automatico, lo *skinning automatico* è stato implementato in Blender seguendo la stessa logica di corrispondenza tra ossa e vertici, ma attraverso uno script Python dedicato che trasferisce i pesi da un modello sorgente già pesato a un oggetto target. Lo script esegue le seguenti operazioni principali:

1. **Selezione delle ossa rilevanti:** vengono mantenuti esclusivamente i gruppi di pesi relativi alla parte inferiore del corpo, coerenti con lo scheletro generale nella fase precedente.

Algorithm 5: DefineBonesToTransfer

Output: bonesToKeep`// Definisce l'insieme delle ossa rilevanti``// utilizzate durante il trasferimento``bonesToKeep $\leftarrow$ { Hips, LeftUpLeg, LeftLeg, LeftFoot, RightUpLeg,
RightLeg, RightFoot };`

2. **Copia dei gruppi di vertici:** per ogni osso selezionato, lo script copia i pesi corrispondenti a ciascun vertice, limitandosi ai valori effettivamente significativi (pesi maggiore di zero).
3. **Creazione di nuovi gruppi di influenza:** per la mesh target, vengono generati automaticamente nuovi *vertex group* corrispondenti alle ossa selezionate, garantendo la compatibilità con il rig del nuovo modello.

Estratto del codice Python . Un frammento rappresentativo dello script utilizzato in Blender mostra la copia dei gruppi di vertici e la creazione di nuovi gruppi di influenza

Algorithm 6: OptimizedWeightTransfer

```

Input: plate, sourceMesh, bonesToKeep

// Verifica che gli oggetti esistano
if plate non esiste or sourceMesh non esiste then
    raise Exception("Mesh non trovata!");

// Cancella eventuali vertex group già presenti sulla mesh del
    piatto
ClearVertexGroups(plate);

// Copia solo i gruppi delle ossa selezionate
foreach vg  $\in$  sourceMesh.vertexGroups do
    if vg.name  $\in$  bonesToKeep then
        // Crea un nuovo gruppo di vertici sulla mesh del piatto
        newVG  $\leftarrow$  plate.AddVertexGroup(vg.name);
        // Copia i pesi dei vertici dalla mesh sorgente
        foreach vertice i  $\in$  sourceMesh.vertices do
            if Weight(vg, i)  $> 0$  then
                AddWeight(newVG, i, Weight(vg, i));

```

Questo frammento mostra come il sistema filtra automaticamente i pesi pertinenti e li assegna ai vertici corrispondenti del nuovo modello. Il risultato è una **mappatura coerente tra ossa e mesh**, che consente di preservare la qualità delle deformazioni anche in presenza di una topologia differenti tra i due oggetti.

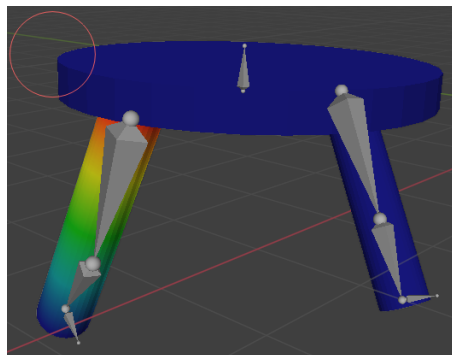


Figura 3.13: Pesi assegnati alla mesh

3.9.2 Collegamento mesh-scheletro e gestione delle deformazioni

Dopo il trasferimento dei pesi, la mesh viene collegata al rig mediante un modificatore di tipo *Armature*, che permette a Blender di calcolare in tempo reale la deformazione dei vertici in base ai movimenti delle ossa. Questo collegamento è del tutto automatico e viene completato senza intervento manuale, risultando in un processo di skinning completamente automatizzato.

L'efficacia del metodo è stata verificata attraverso test di animazione, applicando cicli di camminata e movimenti dinamici. I risultati mostrano che la deformazione delle gambe avviene in maniera stabile e coerente, con una buona trasmissione del movimento anche su modelli privi di articolazioni naturali.

3.10 Retargeting su oggetti inanimati

Dopo la fase di rigging e di skinning automatico, in cui è stato generato e collegato lo scheletro al modello tridimensionale del piatto, la fase successiva ha riguardato l'**applicazione del movimento animato**. In particolare, l'obiettivo è stato quello di **trasferire un'animazione umanoide preesistente**, proveniente da una sorgente esterna (come Mixamo), al modello personalizzato realizzato in Blender.

Questa operazione, nota come **retargeting dell'animazione**, consiste nel mappare i movimenti di un rig sorgente su un rig di destinazione avente una struttura scheletrica diversa, ma compatibile a livello gerarchico. Nel contesto di questo lavoro, il retargeting ha permesso di riutilizzare animazioni di camminata o corsa tipiche di personaggi umanoidi per animare oggetti non convenzionati, come il *piatto* o la *tazzina con gambe*, generando un effetto dinamico coerente senza dover ricorrere a un'animazione manuale.

Il processo è stato sviluppato interamente in **Blender**, attraverso **script Python dedicati**, che automatizzano la fase di assegnazione dei vincoli di trasformazione tra rig sorgente e rig target. Questa automazione ha reso possibile un trasferimento accurato delle animazioni anche in presenza di differenze morfologiche significative tra i modelli, mantenendo la coerenza strutturale e la stabilità del movimento. I

passaggi specifici per questo processo di retargeting su oggetti inanimati sono riassunti nell'Algorithm 7.

Algorithm 7: RetargetAnimation

```

Input: sourceRig, targetRig

// sourceRig:  armatura sorgente, targetRig:  armatura di
//            destinazione
foreach bone  $t$  in targetRig do
    // Itera su tutte le ossa del rig di destinazione
     $s \leftarrow \text{sourceRig.getBone}(t.\text{name});$ 
    // Ricerca dell'osso omonimo nel rig sorgente
    if  $s$  exists then
        apply CopyLocation( $s \rightarrow t$ );
        // Copia la posizione dell'osso sorgente sull'osso
        //      target
        apply CopyRotation( $s \rightarrow t$ );
        // Copia la rotazione dell'osso sorgente sull'osso
        //      target
  
```

3.10.1 Trasferimento di animazioni umanoidi agli oggetti

Per permettere al modello non antropomorfo di riprodurre i movimenti di un rig umanoide (nel caso specifico, un'animazione di camminata proveniente da Mixamo), è stato realizzato un sistema di **retargeting automatico** all'interno di Blender tramite script Python.

L'obiettivo è mappare le ossa del rig sorgente (umanoide) su quelle di rig target (nel nostro caso il *PiattoRig*), trasferendo posizioni e rotazioni delle articolazioni principali. Questo processo è stato implementato utilizzando **vincoli di trasformazione** (constraints) che permettono di copiare in tempo reale i movimenti del rig originale.

In particolare, per ciascun osso del rig del piatto, lo script aggiunge vincoli principali:

- **Copy Location**, per copiare la posizione locale dell'osso corrispondente;
- **Copy Rotation**, per copiare la rotazione, mantenendo la coerenza con il sistema di coordinate locale.

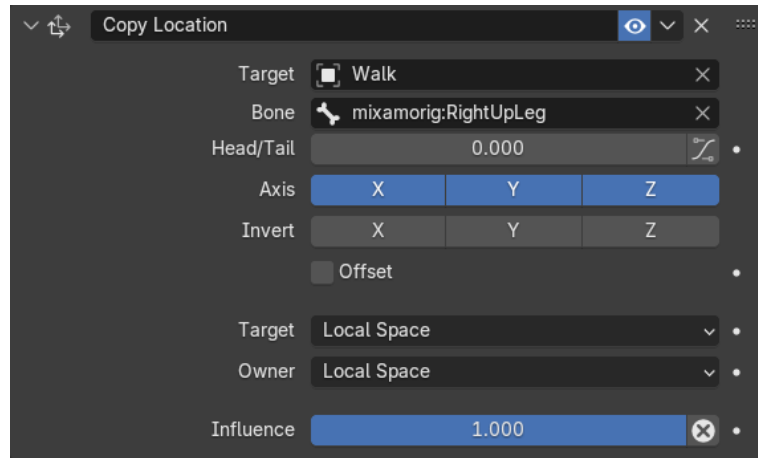


Figura 3.14: Vincoli Copy Location su Blender

L'approccio consente di sincronizzare i movimenti dell'intero rig, evitando la necessità di keyframe manuali o conversioni di animazioni tra sistemi scheletrici differenti. Un frammento esemplificativo del codice implementato è mostrato di seguito:

```
for bone in target.pose.bones:
    src_bone = source.pose.bones.get(bone.name)
    if not src_bone:
        continue

    c_loc = bone.constraints.new(type='COPY_LOCATION')
    c_loc.target = source
    c_loc.subtarget = bone.name
    c_loc.owner_space = 'LOCAL'
    c_loc.target_space = 'LOCAL'

    c_rot = bone.constraints.new(type='COPY_ROTATION')
```

```
c_rot.target = source
c_rot.subtarget = bone.name
c_rot.owner_space = 'LOCAL'
c_rot.target_space = 'LOCAL'
```

Script 1: Script per la copia dei vincoli di posizione e rotazione tra armature.

Questo frammento mostra la parte centrale dello script, in cui ad ogni osso viene assegnata una relazione diretta di copia di posizione e rotazione rispetto all'omologo del rig sorgente. Il risultato è un trasferimento fedele del movimento, applicato anche a oggetti con una morfologia completamente diversa da quella del modello originale.

3.10.2 Gestione del movimento coerente (root motion o offset spaziali)

Per garantire che l'oggetto animato mantenga un movimento coerente nello spazio, è stata introdotta una seconda fase di **controllo del root motion**, ovvero della traslazione globale del rig.

Il problema principale riscontrato riguarda l'osso radice (*mixamorig:Hips*), che un rig umanoide controlla lo spostamento complessivo del corpo. Nel caso di un oggetto statico come il piatto, la riproduzione diretta di tale movimento avrebbe causato uno spostamento dell'intero modello lungo l'asse verticale.

Algorithm 8: ApplyRootMotionFiltering

```

Input: sourceHips, targetHips

// sourceHips: osso dei fianchi dell'armatura sorgente
// targetHips: osso dei fianchi dell'armatura di destinazione
CopyRotation(sourceHips → targetHips);
// Copia la rotazione dell'osso delle anche dalla sorgente al
    target
CopyLocation(sourceHips → targetHips);
// Copia la traslazione dell'osso delle anche dalla sorgente
    al target
DisableZAxisTranslation(targetHips);
// Disabilita la traslazione lungo l'asse verticale (Z) sul
    target, evitando movimenti indesiderati in altezza

```

Per risolvere questo problema, è stato utilizzato un secondo script che gestisce selettivamente il trasferimento delle trasformazioni, limitando il movimento all'asse orizzontale (X e Y) e impedendo la traslazione lungo l'asse Z. La logica dell'algoritmo è riassunta nell'Algorithm 8, che ne evidenzia i passaggi principali.

```

hips = target.pose.bones.get("mixamorig:Hips")
if hips:
    c_loc = hips.constraints.new(type='COPY_LOCATION')
    c_loc.target = source
    c_loc.subtarget = "mixamorig:Hips"
    c_loc.owner_space = 'WORLD'
    c_loc.target_space = 'WORLD'
    c_loc.use_z = False # non copiare l'altezza (Z)

    c_rot = hips.constraints.new(type='COPY_ROTATION')
    c_rot.target = source
    c_rot.subtarget = "mixamorig:Hips"

```

Script 2: Script che cerca di evitare le deformazioni quando viene trasferito il movimento.

Questa configurazione consente di preservare il **movimento orizzontale** del rig, mantenendo però fisso il punto di contatto del modello con il suolo. Il risultato è un'animazione fluida e coerente, dove il piatto segue la dinamica della camminata umanoide senza perdere l'allineamento spaziale.

Capitolo 4

Esperimenti

Il presente capitolo è dedicato all’analisi dei risultati ottenuti applicando la pipeline descritta nel Capitolo 3 e al confronto tra i diversi approcci e algoritmi considerati. Al fine di evitare ridondanze, in questa sezione vengono riportate esclusivamente le informazioni necessarie all’interpretazione dei risultati sperimentali, mentre i dettagli metodologici relativi al rigging, allo skinning e al retargeting automatico sono stati già discussi nei capitoli precedenti.

Gli esperimenti sono stati condotti su oggetti 3D inanimati e non antropomorfi, utilizzando le stesse animazioni di input per tutti gli approcci, così da garantire condizioni di confronto omogenee. I risultati sono presentati principalmente sotto forma di confronti visivi e di una valutazione quantitativa basata su una metrica di errore definita ad hoc, finalizzata a misurare il grado di discostamento da un risultato ritenuto ottimale.

Nelle successive vengono analizzati separatamente i risultati delle diverse fasi di pipeline (rigging, skinning e retargeting) e viene infine fornito un confronto complessivo tra gli approcci considerati.

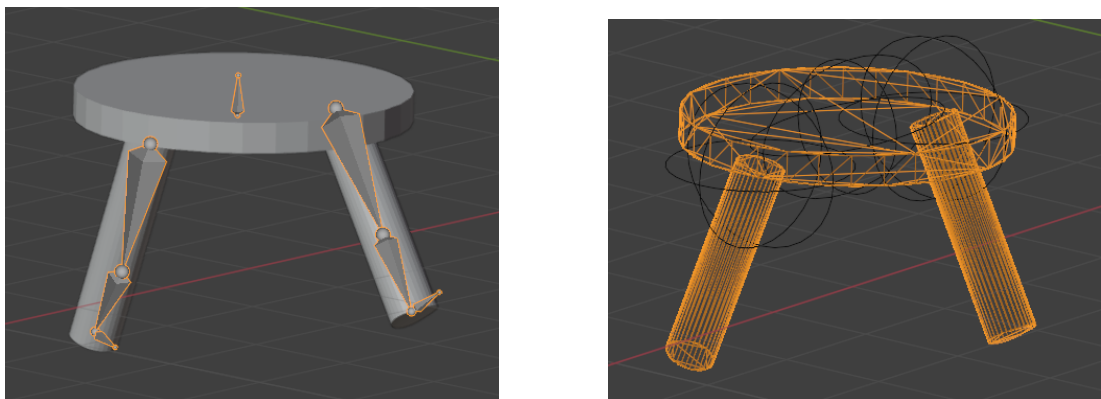
4.1 Risultati del rigging automatico

In questa sezione vengono presentati i risultati ottenuti nella fase di rigging automatico applicando gli approcci UniRig e RigAnything agli oggetti 3D inanimati descritti nel Capitolo 3. Sono stati considerati esclusivamente questi due metodi in quanto entrambi includono una fase di generazione automatica dello scheletro a partire dalla mesh,

rendendo possibile un confronto diretto.

Per ciascun oggetto, il rig generato è stato analizzato dal punto di vista del posizionamento delle ossa, della coerenza strutturale con la geometria della mesh e della copertura delle parti articolabili.

4.1.1 Confronto visivo del rigging



(a) Rigging generate da UniRig

(b) Rigging generate da RigAnything

Figura 4.1: Risultato del rigging automatico

La figura 4.1 mostra il confronto visivo tra le strutture di rigging generate da UniRig e RigAnything sul modello del piatto con due gambe. Nel caso di UniRig, lo scheletro generato è rappresentato da un'armature esplicita, con ossa chiaramente allineate alle principali componenti strutturali dell'oggetto, in particolare alle due gambe e alla posizione centrale del piatto.

Per RigAnything, la struttura di controllo non è espressa tramite un'armature tradizionale, ma attraverso controlli impliciti che guidano la deformazione della mesh. Tali controlli sono visualizzati mediante punti di controllo sovrapposti alla mesh e tramite la visualizzazione wireframe, evidenziandole regioni soggette a movimento.

4.1.2 Analisi qualitativa

L'analisi qualitativa dei risultati evidenzia come UniRig e RigAnything adottino strategie differenti nella fase di rigging automatico, ottimizzando aspetti diversi del problema.

Dal punto di vista strutturale, UniRig genera un'armatura esplicita, semanticamente interpretabile e direttamente utilizzabile all'interno delle pipeline di animazione tradizionali basati su scheletri. La presenza di ossa chiaramente definite facilita le successive fasi di skinning e retargeting, risultando particolarmente vantaggiosa in contesti in cui è richiesto un controllo esplicito della struttura dell'oggetto.

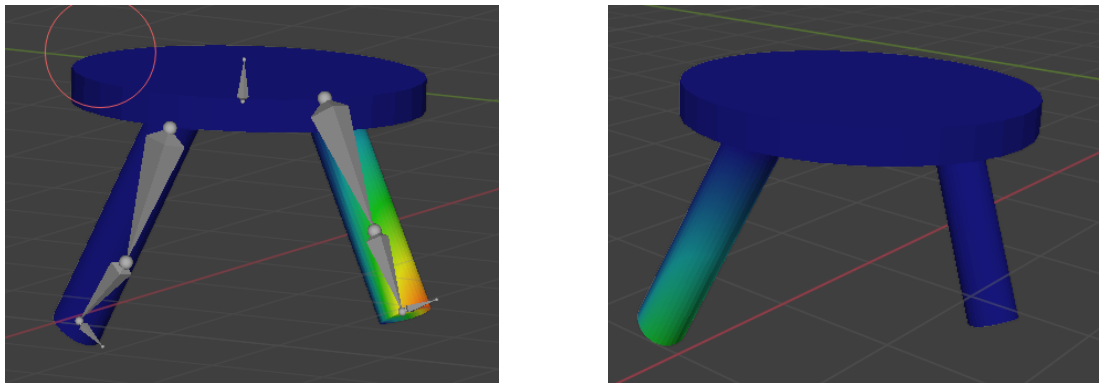
RigAnything, invece, non produce uno scheletro esplicito, ma utilizza una struttura di controllo implicita che privilegia la stabilità della deformazione della mesh. Questo approccio consente una maggiore flessibilità nel trattamento di geometrie non antropomorfe, ma riduce l'interpretabilità del rig e la sua integrazione diretta con strumenti di animazione scheletrica standard.

In sintesi, UniRig risulta più adatto a scenari in cui è richiesta una rappresentazione strutturale chiara del rig, mentre RigAnything mostra vantaggi nella gestione della deformazione, soprattutto in presenza di geometrie articolate e non convenzionali. Nessuno dei due approcci risulta pertanto globalmente superiore, ma ciascuno presenta punti di forza specifici in funzione del criterio di valutazione considerato.

4.2 Risultato dello skinning

In questa sezione vengono analizzati i risultati ottenuti nella fase di skinning automatico valutando la qualità delle deformazioni prodotte durante l'animazione degli oggetti 3D. Lo skinning è stato applicato agli scheletri e alle strutture di controllo generati nella fase di rigging, mantenendo invariata l'animazione di input per tutti gli approcci.

4.2.1 Confronto visivo delle deformazioni



(a) Skinning generate da UniRig

(b) Skinning generate da RigAnything

Figura 4.2: Risultato dello skinning automatico

La figura 4.2 mostra un confronto visivo delle deformazioni ottenute applicando lo skinning automatico ai rig generati da UniRig e RigAnything sul modello del piatto con due gambe, in corrispondenza di frame rappresentativi dell’animazione.

Nel caso di UniRig, la deformazione della mesh risulta coerente con la struttura scheletrica esplicita, ma sono visibili lievi artefatti locali in prossimità delle giunzioni tra le gambe e il piatto, dovuti alla distribuzione dei pesi di influenza sui vertici.

Per RigAnything, la deformazione appare più uniforme lungo le parti articolate, con una riduzione degli artefatti locali nelle regioni di maggiore movimento. Tuttavia, la mancanza di una struttura ossea esplicita rende meno immediata l’individuazione delle regioni di controllo responsabili della deformazione.

4.2.2 Analisi qualitativi dello skinning

L’analisi quantitativa evidenzia come la qualità dello skinning sia fortemente influenzata dalla struttura di rigging sottostante.

Nel caso di UniRig, la presenza di un’armature esplicita consente una chiara associazione tra ossa e regioni della mesh, facilitando il controllo della deformazione e l’eventuale intervento correttivo. Tuttavia, lo skinning automatico risulta più sensibile alla complessità geometrica, mostrando artefatti locali in corrispondenza di giunzioni non perfettamente allineate alla struttura scheletrica.

RigAnything, utilizzando una struttura di controllo implicita, privilegia la stabilità complessiva della deformazione, producendo risultati visivamente più uniformi nelle regioni articolate. Questo approccio riduce la presenza di collassi locali e stretching eccessivo, ma sacrifica parzialmente l'interpretabilità del processo di skinning.

In sintesi, UniRig offre un maggiore controllo strutturale dello skinning, risultando vantaggioso in scenari in cui è richiesta un'animazione scheletrica tradizionale, mentre RigAnything mostra una maggiore robustezza nella deformazione automatica di oggetti 3D non antropomorfi.

4.3 Risultato del retargeting delle animazioni

In questa sezione vengono analizzati i risultati ottenuti nella fase di retargeting delle animazioni, valutando la capacità dei diversi approcci di trasferire correttamente il movimento da una sorgente animata a oggetti 3D inanimati. Le animazioni di input, provenienti da Mixamo o MoveAI, sono state applicate utilizzando la stessa pipeline per tutti i metodi, così da garantire condizioni di confronto omogenee.

4.3.1 Confronto visivo del retargeting

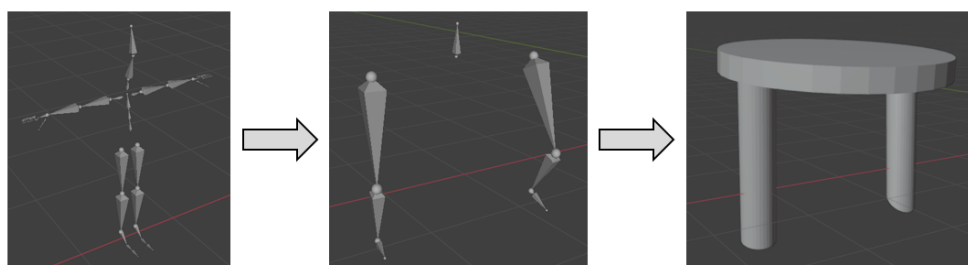


Figura 4.3: Frame rappresentativo del risultato di retargeting dell'animazione sull'oggetto target ottenuto con UniRig

La figura 4.3 mostra un confronto visivo del retargeting dell'animazione sul modello del piatto con due gambe, in corrispondenza di frame rappresentativi del movimento.

Nel caso di UniRig, il retargeting risulta coerente con la struttura scheletrica esplicita generata nella fase di rigging. Il movimento delle gambe segue in modo interpretabile

l'animazione sorgente, mantenendo una chiara corrispondenza tra ossa sorgente e ossa target. Tuttavia, in alcuni frame sono osservabili limitazioni nella naturalezza del movimento, dovute alla rigidità imposta dalla struttura scheletrica.

Per RigAnything, il movimento risulta più fluido e continuo lungo le parti articolate dell'oggetto, con una riduzione delle discontinuità temporali. Tuttavia, l'assenza di una mappatura scheletrica esplicita rende meno immediato il controllo semantico del movimento e la sua modifica manuale.

4.3.2 Analisi qualitativa

L'analisi qualitativa dei risultati evidenzia come la fase di retargeting sia fortemente influenzata dalle scelte effettuate nelle fasi di rigging e skinning.

Tali differenze emergono in modo evidente anche dall'analisi qualitativa delle traiettorie dei punti articolari equivalenti, riportate nei grafici seguenti, in cui è possibile osservare un comportamento più regolare e strutturalmente coerente nel caso di UniRig rispetto a un andamento più fluido ma meno controllabile nel caso di RigAnything.

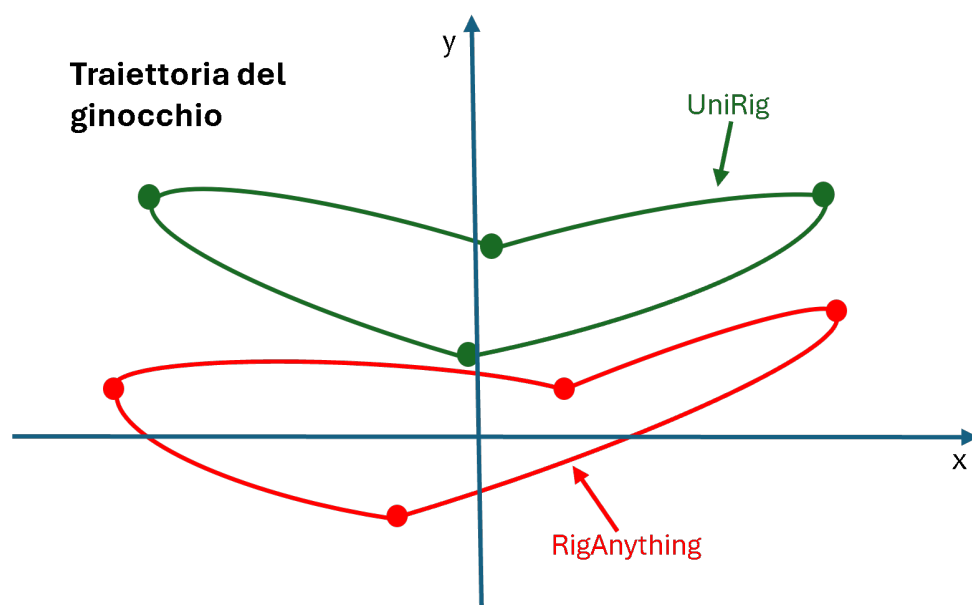


Figura 4.4: Rappresentazione qualitativa delle traiettorie del punto articolare intermedio (equivalente al ginocchio) durante il retargeting dell'animazione, proiettate su un piano bidimensionale.

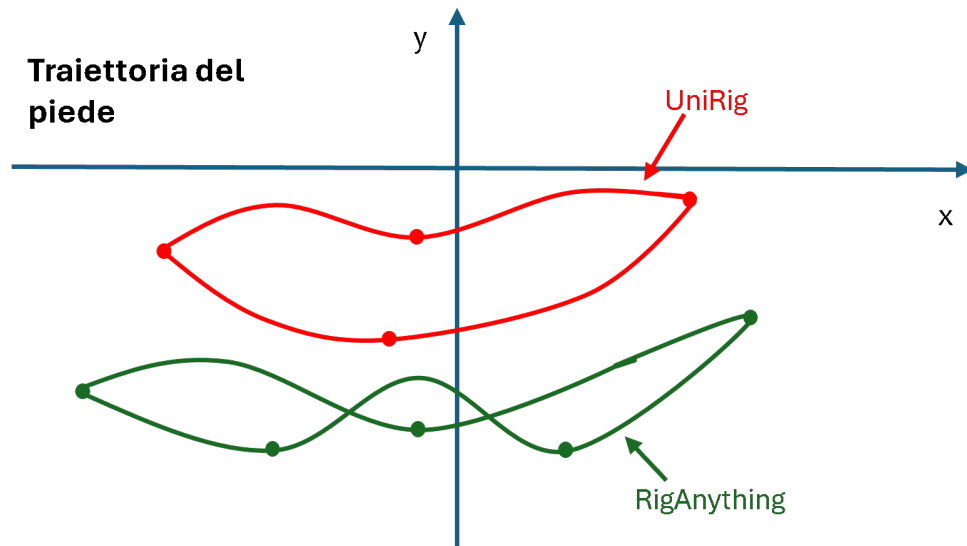


Figura 4.5: Rappresentazione qualitativa delle traiettorie del punto terminale della gamba (equivalente al piede) durante il retargeting dell'animazione.

UniRig, grazie alla presenza di un'armature esplicita, consente un retargeting più diretto e semanticamente coerente con le animazioni sorgente, risultando particolarmente adatto a pipeline basate su scheletri standard. Questa caratteristica facilita il trasferimento di animazioni provenienti da dataset antropomorfi, come Mixamo, anche su oggetti non convenzionali.

RigAnything, invece, mostra una maggiore flessibilità nel trasferimento del movimento, adattando l'animazione alla geometria dell'oggetto senza richiedere una corrispondenza scheletrica rigida. Questo approccio produce movimenti visivamente più fluidi, ma rende più complesso il controllo fine del retargeting e la sua riproducibilità su animazioni differenti.

In sintesi, UniRig risulta più indicato in scenari in cui è richiesta una corrispondenza strutturale tra animazioni sorgente e target, mentre RigAnything privilegia la qualità visiva del movimento adattato, a scapito della controllabilità semantica.

4.3.3 Confronto con altri approcci

Oltre al confronto sperimentale diretto tra UniRig e RigAnything, i risultati ottenuti sono stati messi in relazione con alcuni approcci rappresentativi presenti in letteratura,

tra cui ASMR, AnyTop e NOR (Neural Object Rigging). Poiché tali metodi non sono stati implementati e testati direttamente nel presente lavoro, il confronto è stato condotto su base metodologica e qualitativa, facendo riferimento alle caratteristiche dichiarate e ai risultati riportati nei rispettivi lavori.

Gli approcci considerati differiscono significativamente per quanto riguarda la rappresentazione del rig, la gestione dello skinning e il supporto al retargeting delle animazioni. In particolare, ASMR si basa su una struttura scheletrica esplicita e su una corrispondenza rigida tra sorgente e target, risultando maggiormente orientato a modelli antropomorfi. AnyTop e NOR, invece, adottano strategie più flessibili, mirate al trattamento di geometrie generiche e non convenzionali, ma presentano un supporto limitato o parziale al retargeting di animazioni complesse.

Nel contesto del presente lavoro, UniRig e RigAnything si collocano come approcci intermedi tra i metodi puramente scheletrici e quelli completamente impliciti. UniRig condivide con ASMR l'uso di un'armatura esplicita, facilitando il retargeting semantico delle animazioni, ma estende tale paradigma a oggetti non antropomorfi. RigAnything, invece, si avvicina maggiormente ad approcci come AnyTop e NOR, privilegiando l'adattamento del movimento alla geometria dell'oggetto e la qualità visiva del risultato finale, a scapito della controllabilità strutturale. Questo confronto si vede più chiaramente nella seguente figura 4.6.

Metodo	Skeleton esplicito	Skining automatico	Retargeting	Oggetti inanimati
UniRig	✓	✓	✓	✓
RigAnything	✗	✓	✓	✓
ASMR	✓	⚠	✓	✗
AnyTop	✗	✓	✗	✓
NOR	✗	✓	⚠	✓

Figura 4.6: Confronto qualitativo tra i diversi approcci, evidenziandone punti di forza e limitazioni nel contesto del retargeting di animazioni su oggetti 3D

4.4 Definizione della metrica di errore

4.4.1 Motivazioni e obiettivi

La valutazione dei risultati ottenuti mediante tecniche di rigging automatico, skinning e retargeting di animazioni su oggetti 3D inanimati risulta particolarmente complessa, in quanto non esiste un riferimento univoco che definisca una soluzione ottimale. La qualità del risultato dipende infatti da molteplici fattori, tra cui la coerenza strutturale del rig, la stabilità delle deformazioni e la fedeltà del movimento trasferito. Alla luce di tali considerazioni, è stata definita una metrica di errore composita, con l'obiettivo di quantificare in modo coerente il grado di discostamento del risultato ottenuto rispetto a una soluzione ritenuta ottimale nel contesto applicativo considerato. La metrica proposta consente di confrontare approcci basati su paradigmi differenti, pur mantenendo una valutazione uniforme.

4.4.2 Definizione della metrica

La metrica di errore proposta, denominata **Composite Retargeting Error** (CRE), è definita come una combinazione pesata di quattro componenti principali:

$$CRE = w_s E_s + w_d E_d + w_t E_t + w_c E_c$$

dove E_s, E_d, E_t ed E_c rappresentano rispettivamente l'errore strutturale, l'errore di deformazione, l'errore temporale e l'errore di controllabilità, mentre w_s, w_d, w_t e w_c sono i pesi associati a ciascuna componente, tali che:

$$w_s + w_d + w_t + w_c = 1$$

Ciascuna componente è normalizzata nell'intervallo $[0,1]$, dove il valore 0 indica una soluzione ottimale rispetto al criterio considerato, mentre il valore 1 rappresenta una forte deviazione dalla soluzione desiderata.

4.4.3 Componenti della metrica

Errore strutturale(E_s)

L'errore strutturale misura il grado di coerenza del rig generato rispetto a una struttura scheletrica semanticamente interpretabile. In particolare, vengono valutati la presenza di uno scheletro esplicito, la coerenza tra ossa e parti dell'oggetto e la stabilità della gerarchia del rig. Valori ridotti di E_s indicano una struttura facilmente interpretabile e controllabile, mentre valori elevati sono associati a rig impliciti o difficilmente modificabili.

Errore di deformazione(E_d)

L'errore di deformazione quantifica la qualità delle deformazioni della mesh durante l'animazione, tenendo conto di fenomeni quali stretching eccessivo, collassi locali e intersezioni geometriche. La valutazione è stata condotta mediante analisi visiva dei risultati e, ove disponibile, mediante l'osservazione delle mappe di peso associate allo skinning.

Errore temporale(E_t)

L'errore temporale misura la coerenza del movimento nel tempo, considerando la presenza di discontinuità, jitter o perdita di fluidità nel passaggio tra pose successive. La valutazione è stata effettuata su frame rappresentativi dell'animazione, selezionati in corrispondenza di pose significative.

Errore di controllabilità(E_c)

L'errore di controllabilità valuta il grado di accessibilità e modificabilità del risultato ottenuto. In particolare, vengono considerate la possibilità di intervenire manualmente sul rig, la chiarezza del controllo offerto all'utente e la riutilizzabilità dell'animazione su modelli differenti.

4.4.4 Applicazione della metrica ai metodi sperimentati

La metrica di errore è stata applicata ai metodi UniRig e RigAnything, per i quali sono stati ottenuti risultati sperimentali nello stesso contesto applicativo. I valori riportati sono il risultato di una valutazione qualitativa normalizzata, basata sull'analisi visiva dei risultati e sui criteri definiti in precedenza.

Tabella 4.1: Confronto dei valori della metrica di errore per i metodi sperimentati

Metodo	E_s	E_d	E_t	E_c	CRE
UniRig	0.2	0.4	0.3	0.2	0.28
RigAnything	0.6	0.2	0.2	0.6	0.40

4.4.5 Discussione

L'analisi condotta mediante la metrica proposta evidenzia come gli approcci basati su uno scheletro esplicito tendano a favorire la controllabilità e la coerenza strutturale, mentre i metodi a controllo implicito mostrano generalmente migliori prestazioni in termini di deformazione e adattabilità geometrica. Tali risultati confermano l'assenza di una soluzione globalmente ottimale e sottolineano la presenza di compromessi tra i diversi criteri considerati.

Conclusioni e sviluppi futuri

Questo lavoro di tesi ha affrontato il problema del rigging, skinning e retargeting automatico di animazioni su oggetti 3D inanimati, con particolare attenzione all'adattamento di animazioni antropomorfe a geometrie non convenzionali. L'obiettivo principale è stato quello di analizzare e confrontare diversi approcci automatici, valutandone l'efficacia all'interno di una pipeline basata su Blender.

I risultati ottenuti mostrano come le scelte effettuate nelle fasi di rigging e skinning influenzino in modo significativo la qualità e la controllabilità del retargeting delle animazioni. In particolare, l'approccio basato su UniRig si è dimostrato efficace nel mantenere una corrispondenza strutturale e semantica con le animazioni sorgente, facilitando il riutilizzo di dataset standard come Mixamo anche su oggetti non antropomorfi.

Al contrario, approcci più flessibili come RigAnything tendono a privilegiare la qualità visiva del movimento adattato alla geometria dell'oggetto, a scapito di una maggiore difficoltà nel controllo fine del retargeting e nella riproducibilità dei risultati su animazioni differenti.

Il presente lavoro presenta alcuni limiti. In primo luogo, la valutazione dei risultati è stata prevalentemente qualitativa, concentrandosi sull'analisi visiva e sulla coerenza del movimento, senza introdurre metriche quantitative standardizzate. Inoltre, i test sono stati condotti su un numero limitato di modelli e animazioni, scelti come casi di studio rappresentativi ma non esaustivi.

Un ulteriore limite riguarda l'integrazione parziale di alcuni approcci automatici, che non ha consentito una valutazione uniforme di tutte le fasi della pipeline per ciascun metodo considerato.

Possibili sviluppi futuri di questo lavoro includono l'introduzione di metriche quantita-

tive per la valutazione del retargeting, come l'analisi delle traiettorie dei punti articolari o la misurazione della deviazione rispetto a movimenti di riferimento. Tali metriche permetterebbero di affiancare all'analisi qualitativa una valutazione più oggettiva dei risultati.

Un ulteriore sviluppo potrebbe consistere nell'estensione dei test a un numero maggiore di modelli 3D e categorie di oggetti, al fine di valutare la robustezza degli approcci considerati in scenari più eterogenei

Infine, l'integrazione di tecniche di apprendimento automatico per l'ottimizzazione automatica del rigging e del retargeting potrebbe rappresentare una direzione promettente per migliorare ulteriormente l'adattamento delle animazioni a geometrie non convenzionali.

Bibliografia

- [Abh24] Shoubhit Abhin. The Importance of Quaternions. *TomRocksMaths Competition*, 4:2, 3, 2024.
- [Bus20] Vannevar Bush. How to represent 3d data?, 2020.
- [FRGD24] Ricardo García-Salcedo Fernando Ricardo González Díaz, Vicent Martinez Badenes. Educational perspectives on quaternions: Insights and applications. pages 14, 15, 16, 2024.
- [JPZ25] MENG-HAOGUO YAN-PEI CAO SHI-MIN HU JIA-PENG ZHANG, CHENG-FENG PU. One Model to Rig Them All: Diverse Skeleton Rigging with UniRig. *Visual Media Lab, KAIST*, page 8, 2025.
- [Mar19] Leonardo Marini. *Tecniche di animazione 3D nella realizzazione di un cortometraggio*. PhD thesis, Università di Bologna, 2018/2019.
- [NLG23] Zebin Xu Ying Li Na Lei, Zezeng Li and Xianfeng Gu. What’s the situation with intelligent mesh generation: A survey and perspectives. *Artificial Intelligence (cs.AI)*, pages 3 – 4, 2023.
- [SH25] Chaelin Kim Sihun Cha Junyong Noh Seokhyeon Hong, Soojin Choi. ASMR:Adaptive Skeleton-Mesh Rigging and Skinning via 2D Generative Prior. *Visual Media Lab, KAIST*, page 1, 2025.
- [Tag20] Timothy Jeruzalski+ David I.W. Levin+ Alec Jacobson+Paul Lalonde Mohammad Norouzi Andrea Tagliasacchi. Nilbs: Neural inverse linear blend skinning. *Google Research*, 2020.
- [Whi] J. (s.d.). White. *Trasformazioni geometriche 2D/3D*. O’Reilly.